STUDY NOTES / COMPLETE TUTORIAL

Python
全教程复习笔记

知识梳理 · 代码答案 · 逐步解析

基于廖雪峰《Python 教程》 | 112 个章节页面

知识前置路线

01 环境与核心语法 — 类型 · 容器 · 控制流 · 函数

↓

02 抽象与程序组织 — 迭代 · 闭包 · 装饰器 · 模块 · 类

↓

03 可靠地处理数据 — 异常测试 · 文件 · 编码 · 正则 · 标准库

↓

04 完成实际应用 — 并发 · 图形 · 网络 · 邮件 · 数据库

↓

05 Web 与异步 IO — HTTP · WSGI · 模板 · asyncio · aiohttp

重点识记 黄色记规则，蓝色读方法，红色查易错，绿色做验收。

46 组原练习答案 · 29 组讲解示例 · 42 项知识清单

原教程作者：廖雪峰；本笔记重组知识、改写题意并补充解析。

整理日期：2026-11-09 | CC BY-NC-SA 4.0

目录与章节导航

01 认识 Python 与运行环境 4

02 基础类型、容器与控制流 5

03 函数、参数与递归 10

04 切片、迭代、生成式与生成器 13

05 函数式编程与装饰器 16

06 模块与面向对象 21

07 面向对象高级特性 24

08 异常、调试与测试 28

09 文件、目录与序列化 30

10 多进程、多线程与分布式任务 33

11 正则表达式 36

12 常用内建模块：时间与数据组织 38

13 常用内建模块：二进制、摘要与资源管理 40

14 常用内建模块：HTTP、XML、HTML 与 venv 46

15 第三方库与图形界面 50

16 TCP、UDP 与电子邮件 53

17 数据库、SQL 与 ORM 58

18 HTTP、HTML、WSGI 与模板 61

19 协程、asyncio 与 aiohttp 63

20 FAQ、速查与自测回路 66

附录 A 练习覆盖与来源索引 68

附录 B 代码验证记录 69

附录 C 全章节原文链接 70

基于廖雪峰《Python 教程》 | 知识梳理 · 代码答案 · 逐步解析

阅读说明与知识路线

来源：廖雪峰官方网站《Python 教程》，核对日期 2026-11-09。覆盖目录第 1—25 章及全部子节，共 112 个章节页面；将重复的导言合并讲解。原文作者为廖雪峰。本笔记重新组织知识、改写题意并编写参考答案；原文及本演绎笔记依 CC BY-NC-SA 4.0 共享。许可：https://creativecommons.org/licenses/by-nc-sa/4.0/ 。

原教程入口：https://liaoxuefeng.com/books/python/introduction/index.html

重点 使用顺序：先读黄色“重点”，再合上答案口述规则，最后自己运行代码核对。蓝色解释方法，红色标记易错点，绿色标记验收与输出。颜色均配文字标签，打印成灰度后仍能识别。

本文采用 good-learning 的知识清单、前置关系、完整例题、变式与自检方法。你要的是整套复习笔记，因此不虚构考试日期、成绩、每日可用时间或已掌握进度；所有个人掌握状态均为“待自测”。

知识路线图

环境与输入输出 → 类型、容器、分支循环 → 函数与参数 → 切片、迭代、生成器 → 高阶函数、闭包、装饰器 → 模块与类 → 异常与测试 → 文件与序列化 → 并发、正则、标准库 → 图形、网络、邮件、数据库 → Web 与异步 IO。

前置关系：列表与函数是生成器的基础；函数对象与作用域是闭包和装饰器的基础；编码、上下文管理与异常是文件及网络的基础；类、SQL 和函数回调支撑 ORM 与 Web 框架。

代码与题号约定

E 开头：原教程练习的题意改写及参考答案；S 开头：原教程讲解示例的整理或配套补充。每段代码是独立片段，不应把不同题目的同名类、同名函数直接拼在同一文件。补全原题代码时保留原题的测试数据。

纯标准库示例以 Python 3.10+ 为语法基线，`match` 需要 3.10+。第三方库示例写明依赖；网络、邮箱、数据库服务和图形环境各有运行条件，不能仅凭代码就保证外部服务可用。附录给出代码验证范围及来源索引。

01 认识 Python 与运行环境

来源章节：1 简介、2 Python 历史、3 安装 Python、3.1 Python 解释器、4 第一个 Python 程序、4.1 使用文本编辑器、4.2 输入和输出。

K01 知识清单 | 应能独立完成什么

- 解释 Python 的动态类型与解释执行；区分语言规范、解释器和编辑器。CPython 是常用实现；PyPy 使用不同执行技术，Jython/IronPython 面向其他运行平台，不能把这些实现当作完全等价的 Python 3 环境。
- 会查看版本、运行 `.py` 文件、进入和退出交互环境；脚本用 UTF-8 保存。Python 由 Guido van Rossum 创建；复习历史重在理解 Python 2 与 3 的兼容性差异，不背版本发布日期。
- 会使用 `print()` 输出、`input()` 输入，并在算术前显式转为 `int` 或 `float`。

重点 终端中的 `python3 hello.py` 是系统命令；交互环境中的 `>>>`、续行提示 `...` 是提示符，不能复制进 `.py` 文件。Python 用缩进表示代码块，建议统一四个空格。

易错 “高级语言”描述抽象层次，不能直接推出“任何任务都简单”或固定的速度倍数。Python 的适用性由库、实现、部署方式和性能需求共同决定；教程中的相对速度与应用限制应作为入门说明理解。

S01 从文本输入到数值计算

```
name = input('Name: ')
age = int(input('Age: '))
print(f'Hello, {name}! Next year: {age + 1}')
```

解析：`input()` 总是返回字符串；`int()` 把合法的整数字符串转为整数；f-string 的花括号中可以计算表达式。输入不是整数时抛出 `ValueError`，第 08 章会讲异常处理。
变式：把年龄改成小数，需选择 `float()`；若输入 `3.5`，不能直接用 `int('3.5')`。

E01 输出乘法等式

题意改写 | 原 4.2：用输出函数同时打印算式和计算结果。

```
print('1024 * 768 =', 1024 * 768)
```

详解：引号内只是文本；引号外的乘法会先求值。`print()` 默认在参数之间插入空格，所以结果为 `1024 * 768 = 786432`。只打印 `'1024 * 768'` 不会进行计算。

验收 能说明“字符串字面量”和“表达式”的区别，并在命令行独立运行脚本。

02 基础类型、容器与控制流

来源章节：5 Python 基础、5.1—5.7。

K02 数值、布尔值与变量

- `int` 支持任意精度整数，但受内存限制；`float` 精度和范围有限，计算可能舍入或溢出。`None` 表示缺少值，检查通常用 `is None`。
- `/` 做真除法；`//` 向负无穷取整；`%` 配合地板除满足 `a == (a // b) * b + a % b`。例如 `-7 // 3 == -3`、`-7 % 3 == 2`。
- `=` 绑定名字，`==` 比较值，`is` 比较对象身份。大写名字表示常量只是约定。赋值不会自动复制对象。
- `and` 与 `or` 会短路，返回参与运算的某个操作数，并非总返回布尔类型；`not` 返回布尔值。

重点 `a = b` 后，两个名字可指向同一个对象。修改共同指向的列表会互相可见；把某个名字重新绑定到另一个对象，不会改变另一个名字的绑定。

S02 赋值、别名与浅复制

```
a = [1, 2]
b = a
c = a.copy()
a.append(3)
assert b == [1, 2, 3]
assert c == [1, 2]
```

解析：`b` 是别名；`c` 是外层新列表。嵌套列表的浅复制仍共享内部对象，需要独立内部数据时再考虑 `copy.deepcopy()`。

E02 变量、引号与原始字符串

题意改写 | 原 5.1：输出整数、浮点数、带引号与多行字符串。

```
n = 123
f = 456.789
s1 = 'Hello, world'
s2 = "Hello, 'Adam'"
s3 = r'Hello, "Bart"'
s4 = r'''Hello,
Bob!'''
print(n, f, s1, s2, s3, s4, sep='\n')
```

详解：`sep='\n'` 让每个参数另起一行。第二个名字两侧的单引号是实际字符；`r` 使反斜杠按原样保留，但本题 `s3` 没有反斜杠，因此双引号正常显示。三引号中的实际换行也被保留，最后输出 `Hello,` 和 `Bob!` 两行。

K03 字符串与编码

- Python 3 的 `str` 表示 Unicode 文本；`bytes` 表示字节。`text.encode('utf-8')` 从文本到字节；`data.decode('utf-8')` 从字节到文本。
- `len('  中文') == 2`，而 UTF-8 编码后字节长度是 6。Unicode 码点个数也不总等于用户看到的字形个数。
- `ord()` 返回单个字符码点，`chr()` 反向转换；`'abc'.upper()`、`'ABC'.lower()` 返回新字符串。
- 格式化可用 `%`、`str.format()` 和 f-string；复习优先掌握 `f'{value:.2f}'`、`f'{ratio:.1%}'`。

易错 文件编码和文件内容是两回事。错误编码可能引发 `UnicodeDecodeError`；`errors='ignore'` 会丢数据，不能当成通用修复方案。原始字符串也不能以奇数个反斜杠结尾。

E03 成绩提升百分比

题意改写 | 原 5.2：成绩从 72 到 85，保留一位小数显示相对增长百分比。

```
s1, s2 = 72, 85
r = (s2 - s1) / s1
print(f'{r:.1%}')
```

详解：增量是 13，基准是原成绩 72，比例为 `13 / 72`。百分比格式会乘 100 并加 `%`，输出 `18.1%`。若先手动乘 100，应改用 `f'{r:.1f}%'`，不能再用百分比格式。

易错 原题把相对增长写作“提升的百分点”；按题目计算意图答案为 18.1%。若把 72% 到 85% 当作两项百分率相减，才是 13 个百分点。基准为 0 时相对增长未定义，应单独处理。

K04 list、tuple、dict 与 set

- `list` 有序可变，支持索引、切片、`append`、`insert`、`pop`；负索引从末尾计数。`tuple` 固定各位置的引用，单元素元组必须带逗号。
- `dict` 是键到值的映射，现代 Python 保留插入顺序；`in d` 判断键，`d.get(k)` 可避免键不存在时报错。键必须可哈希，列表不能作键。
- `set` 保存不重复的可哈希元素，不保证排序；空集合是 `set()`，`{}` 是空字典。交集 `&`，并集 `|`，差集 `-`。
- 字符串和元组不可变，但包含列表的元组内部列表仍可修改；包含不可哈希元素的元组也不可哈希。

E04 嵌套索引与元组辨认

题意改写 | 原 5.3：从三行列表取指定元素，并判断哪些写法构成元组。

```
L = [
    ['Apple', 'Google', 'Microsoft'],
    ['Java', 'Python', 'Ruby', 'PHP'],
    ['Adam', 'Bart', 'Bob']
]
print(L[0][0])
print(L[1][1])
print(L[2][2])
assert isinstance((), tuple)
assert not isinstance((1), tuple)
assert not isinstance([2], tuple)
assert isinstance((3,), tuple)
assert isinstance((4, 5, 6), tuple)
```

详解：第一次索引选外层行，第二次索引选该行元素，均从 0 开始。输出为 Apple、Python、Bob。原题 a、d、e 是元组；b 是整数；c 是列表。决定非空元组的关键是逗号。

K05 条件、模式匹配与循环

- `if/elif/else` 自上而下只执行第一个满足的分支。`0`、空容器、空字符串、`None` 等在条件中为假。
- `match/case` 可匹配常量、序列和结构；`case _` 为兜底，`|` 表示候选，`if` 守卫增加条件。裸名字通常是捕获变量，不能随意当成常量。
- `for` 遍历可迭代对象；`range(start, stop, step)` 左闭右开。`while` 在条件成立时重复。
- `break` 退出最近一层循环；`continue` 跳到下一轮；注意循环状态必须能变化，才能避免意外无限循环。

E05 BMI 分支判断

题意改写 | 原 5.4：身高 1.75 m、体重 80.5 kg，计算 BMI 并输出分类。

```
height, weight = 1.75, 80.5
bmi = weight / height ** 2
if bmi < 18.5:
    result = '过轻'
elif bmi < 25:
    result = '正常'
elif bmi < 28:
    result = '过重'
elif bmi < 32:
    result = '肥胖'
else:
    result = '严重肥胖'
print(f'{bmi:.2f}: {result}')
```

详解：先算平方再做除法，得到约 26.29，落在 `[25, 28)`，输出“过重”。采用“下界包含、上界不包含”的连续分段，因此 25 进入过重，32 进入最后一段。这里仅用于练习条件判断，分类沿用题目。
变式：输入身高时先检查 `height > 0`，否则除法无意义。

E06 逐项问候与无限循环

题意改写 | 原 5.6：依次问候列表中的名字；原文另要求尝试写无限循环。

```
L = ['Bart', 'Lisa', 'Adam']
for name in L:
    print(f'Hello, {name}!')

# Infinite-loop form (do not run during batch checks):
# while True:
#     pass
```

详解：`for` 每轮把一个名字绑定到 `name`，打印三行问候。`while True: pass` 的条件永不改变，因此不会自行结束；在终端按 Ctrl+C 会产生 `KeyboardInterrupt`。批量验证时保留为注释，避免挂起。

S03 模式匹配解析命令

原讲解示例整理 | 5.5。

```
args = ['gcc', 'hello.c', 'world.c']
match args:
    case ['gcc']:
        print('missing source file')
    case ['gcc', first, *others]:
        print('compile:', first, *others)
    case ['clean']:
        print('clean')
    case _:
        print('invalid command')
```

解析：第二个模式要求至少两个元素；第一个文件绑定到 `first`，其余放入列表 `others`。模式只解析结构，并未真正调用编译器。`case x` 会捕获值，不等价于和某个现有变量做相等比较。

> **验收** 能解释切片与索引的边界差异、`{}` 的类型、共享列表的别名，以及分支顺序为什么会影响结果。

03 函数、参数与递归

来源章节：6 函数、6.1 调用函数、6.2 定义函数、6.3 函数的参数、6.4 递归函数。

K06 函数基本规则

- 调用函数需满足参数约定；`abs`、`max`、`int`、`float`、`str`、`bool` 是常用内建函数或类型。
- `def` 创建函数对象；`return` 立即结束本次调用；没有显式返回值时返回 `None`。`pass` 仅表示空语句。
- `return x, y` 返回一个元组，可以用 `a, b = f()` 解包。函数中赋值的名字通常是局部变量。
- 参数检查可用 `isinstance()`；非法值通常抛出 `ValueError`，类型不合适通常抛出 `TypeError`。

E07 十六进制转换

题意改写 | 原 6.1：把 255 与 1000 转换成带前缀的十六进制字符串。

```
print(hex(255))
print(hex(1000))
```

详解：结果分别是 `0xff` 和 `0x3e8`，返回类型为 `str`；原整数并未被修改。若只需要数字部分，可用 `f'{255:x}'`，输出 `ff`。

E08 一元二次方程求根

题意改写 | 原 6.2：定义 `quadratic(a, b, c)` 返回两个实根。

```
import math

def quadratic(a, b, c):
    if a == 0:
        raise ValueError('a must be nonzero')
    delta = b * b - 4 * a * c
    if delta < 0:
        raise ValueError('no real roots')
    root = math.sqrt(delta)
    return (-b + root) / (2 * a), (-b - root) / (2 * a)

assert quadratic(2, 3, 1) == (-0.5, -1.0)
assert quadratic(1, 3, -4) == (1.0, -4.0)
```

详解：先验证方程确为二次方程，再计算判别式 $\Delta = b^2 - 4ac$。当 $\Delta \geq 0$ 时，两个实根为 $x_1 = (-b + \sqrt{\Delta})/(2a)$、$x_2 = (-b - \sqrt{\Delta})/(2a)$。重根时返回两个相同值。这里主动限定实数解，不把负判别式传给 `math.sqrt()`。
变式：若题目允许复数解，应改用 `cmath.sqrt()` 并重新说明返回值约定；若允许 `a=0`，另写一次方程分支。

K07 参数顺序与默认参数

- 位置参数：按顺序绑定；默认参数：调用时可以省略；`*args` 收集额外位置参数为元组；`**kwargs` 收集额外关键字参数为字典。
- `def f(a, b=0, *args, scale=1, **kw)` 中，`scale` 是仅限关键字参数。无 `*args` 时可写 `def f(a, *, scale=1)`。
- 调用时 `f(*items)` 展开序列，`f(**mapping)` 展开字符串键映射；同一个参数重复赋值会报错。

重点 默认值在函数定义时计算一次。需要每次调用新建列表时，用 `None` 作默认值，再在函数体内创建列表。

```
def add_end(items=None):
    if items is None:
        items = []
    items.append('END')
    return items
```

解析： 用 `items is None` 区分“未传入”和“传入空列表”。写 `items = items or []` 会使调用者传来的空列表被替换，改变接口行为。

E09 一个或多个数的乘积

题意改写 | 原 6.3：允许一个以上位置参数；零参数调用必须失败。

```
def mul(x, *args):
    result = x
    for value in args:
        result *= value
    return result

assert mul(5) == 5
assert mul(5, 6) == 30
assert mul(5, 6, 7) == 210
assert mul(5, 6, 7, 9) == 1890
```

详解： 第一个必填参数 `x` 保证最少有一个数；其余数由 `args` 接收。只有一个参数时循环不执行，直接返回它。调用 `mul()` 时参数绑定就会抛出 `TypeError`，符合原题要求。

K08 递归的判断框架

递归必须有明确的终止条件，并使问题规模趋向该条件。每次调用会占用栈空间；常见 CPython 不保证尾递归优化，不能通过“尾递归写法”规避递归深度限制。可用循环或显式栈改写。

E10 汉诺塔

题意改写 | 原 6.4：将 n 个盘从 a 借助 b 移到 c，打印每一步。

```
def move(n, a, b, c):
    if n < 1:
        raise ValueError('n must be positive')
    if n == 1:
        print(a, '-->', c)
        return
    move(n - 1, a, c, b)
    print(a, '-->', c)
    move(n - 1, b, a, c)

move(3, 'A', 'B', 'C')
```

详解：先把上方 n−1 个盘从 a 移到 b，此时 c 是辅助柱；再将最大的盘从 a 移到 c；最后把 b 上的 n−1 个盘借 a 移到 c。递推关系为 $T(n) = 2T(n-1) + 1$，故总步数 $T(n) = 2^n - 1$。三个盘共 7 步：A→C、A→B、C→B、A→C、B→A、B→C、A→C。

> **验收** 不看答案写出 `mul` 和汉诺塔；能解释“定义时收集参数”与“调用时展开参数”的区别。

04 切片、迭代、生成式与生成器

来源章节：7 高级特性、7.1—7.5。

K09 切片与迭代

- `seq[start:stop:step]` 左闭右开，步长不能是 0；`seq[::-1]` 反向取值。切片越界通常会截断，单个索引越界通常抛出 `IndexError`。
- `for x in d` 遍历字典键；`d.values()` 遍历值；`d.items()` 遍历键值对。`enumerate(seq)` 给出索引和值；`zip(a,b)` 默认到最短输入结束。
- `isinstance(x, Iterable)` 常用于检查迭代接口，应从 `collections.abc` 导入。

E11 用切片去除首尾普通空格

题意改写 | 原 7.1：不用 `strip()` 实现 `trim(s)`。

```
def trim(s):
    start, stop = 0, len(s)
    while start < stop and s[start] == ' ':
        start += 1
    while stop > start and s[stop - 1] == ' ':
        stop -= 1
    return s[start:stop]

assert trim('  hello  world  ') == 'hello  world'
assert trim('') == ''
assert trim('    ') == ''
```

详解：用两个边界跳过首尾空格，只切片一次，保留中间空格。条件先检查范围，再访问索引，避免空字符串报错。原题仅要求普通空格；若想处理制表符、换行等空白，才考虑 `isspace()`，那是扩展规则。

E12 迭代求最小值与最大值

题意改写 | 原 7.2：一次遍历列表，空列表返回 `(None, None)`。

```
def findMinAndMax(L):
    if not L:
        return None, None
    low = high = L[0]
    for value in L[1:]:
        if value < low:
            low = value
        if value > high:
            high = value
    return low, high

assert findMinAndMax([]) == (None, None)
assert findMinAndMax([7]) == (7, 7)
assert findMinAndMax([7, 1, 3, 9, 5]) == (1, 9)
```

详解：用第一个实际元素初始化，不能用 0，否则全正数的最小值或全负数的最大值会错误。元素需支持相互比较。比较次数随列表长度线性增长；本写法的 `L[1:]` 会额外复制切片，若输入很大可改用 `iter(L)` 与 `next()` 避免复制。

K10 生成式与惰性计算

- 列表生成式 `[f(x) for x in xs if condition]` 先过滤、再变换；`[a if c else b for x in xs]` 是选择输出，必须有 `else`。
- 圆括号生成器表达式 `(f(x) for x in xs)` 按需计算；含 `yield` 的函数调用时返回生成器，而不是立刻执行完函数体。
- `Iterable` 可提供迭代器；`Iterator` 可用 `next()` 逐个取值，耗尽时抛出 `StopIteration`。列表可反复创建新迭代器，同一生成器通常只能消费一轮。

E13 过滤非字符串再转小写

题意改写 | 原 7.3：混合列表中只处理字符串。

```
L1 = ['Hello', 'World', 18, 'Apple', None]
L2 = [s.lower() for s in L1 if isinstance(s, str)]
assert L2 == ['hello', 'world', 'apple']
```

详解：尾部 `if` 先判定元素是否保留，因此不会对 18 或 `None` 调用不存在的 `lower()`。不应把整数先转成字符串，因为题目要求过滤它们。

E14 杨辉三角生成器

题意改写 | 原 7.4：每次迭代返回下一行列表，可无限生成。

```
from itertools import islice

def triangles():
    row = [1]
    while True:
        yield row
        row = [1] + [a + b for a, b in
                     zip(row, row[1:])] + [1]

rows = list(islice(triangles(), 10))
assert rows[0] == [1]
assert rows[4] == [1, 4, 6, 4, 1]
assert rows[9] == [1, 9, 36, 84, 126, 126, 84, 36, 9, 1]
```

详解：两端固定为 1，中间每项等于上一行相邻两项之和。`yield` 暂停并交出当前行，下次迭代再继续。每轮重新创建列表，因此已经保存的各行不会被后续原地修改；`islice` 限制取 10 行，不能直接 `list(triangles())`。

S04 迭代器的耗尽

```
it = iter([10, 20])
assert next(it) == 10
assert list(it) == [20]
assert list(it) == []
```

解析：`next()` 已消耗第一个元素，第一次 `list()` 取走剩余部分，第二次得到空列表。生成器的 `return value` 结束迭代，并把值放入 `StopIteration.value`；`for` 不会把它当成又一个产出值。

> **验收** 能说明“能用于 for”与“能用于 next”的区别，并能手推杨辉三角下一行。

05 函数式编程与装饰器

来源章节：8 函数式编程、8.1 高阶函数、8.1.1—8.1.3、8.2—8.5。

K11 高阶函数与排序

- 函数可赋给变量、作为参数、作为返回值。`f` 是函数对象，`f()` 是调用结果。
- `map(f, xs)` 按项转换，`filter(p, xs)` 保留条件为真的项；两者返回惰性迭代器。`reduce(f, xs, initial)` 来自 `functools`，把序列累计为一个值。
- `sorted(xs, key=..., reverse=...)` 返回新列表，排序稳定；`list.sort()` 原地排序并返回 `None`。`key` 返回排序依据，不是两个元素的比较结果。

E15 名字规范化

题意改写 | 原 8.1.1 第一问：用 `map()` 将英文名字变为首字母大写、其余小写。

```
def normalize(name):
    return name[:1].upper() + name[1:].lower()

result = list(map(normalize, ['adam', 'LISA', 'barT']))
assert result == ['Adam', 'Lisa', 'Bart']
```

详解：切片允许空字符串，避免 `name[0]` 越界；`map` 对每个名字应用同一个转换。`capitalize()` 对常规英文也能完成，但这里展开步骤以便复习。

E16 reduce 求积

题意改写 | 原 8.1.1 第二问：对一个列表累计相乘。

```
from functools import reduce

def prod(L):
    return reduce(lambda acc, x: acc * x, L, 1)

assert prod([3, 5, 7, 9]) == 945
assert prod([]) == 1
```

详解：累加器初始为乘法单位元 1，依次得到 3、15、105、945。这里约定空积等于 1；若不提供初始值，空列表会抛出 `TypeError`。这与 E09 要求至少一个位置参数的接口不同。

E17 用 map/reduce 解析十进制小数

题意改写 | 原 8.1.1 第三问：不用 `float(s)`，把 `'123.456'` 转成浮点数。

```
from functools import reduce

def str2float(s):
    sign = -1 if s.startswith('-') else 1
    body = s[1:] if s[:1] in ('+', '-') else s
    if body.count('.') > 1:
        raise ValueError('invalid decimal')
    whole, dot, frac = body.partition('.')
    digits = whole + frac
    if not digits or any(c not in '0123456789' for c in digits):
        raise ValueError('invalid decimal')
    number = reduce(lambda a, b: 10 * a + b,
                    map(lambda c: ord(c) - ord('0'), digits), 0)
    return sign * number / (10 ** len(frac))

assert abs(str2float('123.456') - 123.456) < 1e-5
assert str2float('-.5') == -0.5
```

详解：先分离符号与小数点；把每个数字字符映射为整数；用“已有值乘 10 加新数字”合成整数 123456；小数位有三位，因此除以 1000。此实现支持普通带符号十进制写法，不支持指数、空格或下划线；一般业务仍优先使用标准转换函数。

E18 filter 筛选回数

题意改写 | 原 8.1.2：保留正整数中顺读、倒读相同的数字。

```
def is_palindrome(n):
    s = str(n)
    return s == s[::-1]

assert list(filter(is_palindrome, range(1, 30))) == [
    1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 22
]
```

详解：把数字转换为字符串，用反向切片比较。原范围 `range(1, 1000)` 不包含 1000。只取值不保存时可直接遍历 `filter`；若把同一个结果迭代器消费两次，第二次为空。

E19 按姓名、按成绩排序

题意改写 | 原 8.1.3 两问：名字升序，分数降序。

```
L = [('Bob', 75), ('Adam', 92), ('Bart', 66), ('Lisa', 88)]

def by_name(t):
    return t[0]

def by_score(t):
    return -t[1]

print(sorted(L, key=by_name))
print(sorted(L, key=by_score))
```

详解：名字结果依次为 Adam、Bart、Bob、Lisa；成绩结果依次为 Adam、Lisa、Bob、Bart。原题第二问不带 `reverse=True`，因此 `key` 返回负分数，使高分对应更小的键。也可用 `key=lambda t: t[1], reverse=True`，两种方向控制不要重复使用。

K12 闭包、匿名函数与偏函数

- 闭包保存函数及其引用的外层变量；读取的是变量，不是定义时的冻结副本。循环中创建闭包时，可用默认参数固定当轮值。
- `nonlocal` 让内层函数重新绑定最近的外层函数变量；`global` 针对模块级变量。
- `lambda` 只能包含单个表达式；适合短小的 key 或谓词，复杂逻辑用 `def`。
- `functools.partial(func, ...)` 预先绑定部分参数；调用时某些关键字默认值仍可被覆盖。

E20 闭包计数器

题意改写 | 原 8.2：每个新计数器从 1 开始，之后逐次增加。

```
def createCounter():
    count = 0
    def counter():
        nonlocal count
        count += 1
        return count
    return counter

a, b = createCounter(), createCounter()
assert [a(), a(), a()] == [1, 2, 3]
assert b() == 1
```

详解：每次执行外层函数都会建立独立的 `count` 绑定；返回的内层函数使其继续存在。`nonlocal` 允许执行 `count += 1`；没有它会被视为局部变量，读取未赋值的局部变量时出错。

E21 用匿名函数筛选奇数

题意改写 | 原 8.3：将独立的奇数判断函数改写成 lambda。

```
L = list(filter(lambda n: n % 2 == 1, range(1, 20)))
assert L == [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

详解：`lambda` 接收一个数并返回布尔判断；`filter` 保留真值对应的元素。不能写成 `lambda n: return ...`，因为 `return` 不是表达式。

K13 装饰器调用时机

`@decorator` 等价于定义函数后执行 `f = decorator(f)`。装饰器本身在定义时运行；包装函数在每次调用时运行。包装通常用 `*args, **kwargs` 透传参数，用 `functools.wraps` 保留名称与文档，必须返回原函数结果。

E22 函数计时装饰器

题意改写 | 原 8.4 第一问：对任意函数输出一次调用的耗时，保留返回值。

```
import time
from functools import wraps

def metric(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return fn(*args, **kwargs)
        finally:
            elapsed = (time.perf_counter() - start) * 1000
            print(f'{fn.__name__} executed in {elapsed:.3f} ms')
    return wrapper

@metric
def fast(x, y):
    return x + y

assert fast(11, 22) == 33
```

详解：开始时间必须在包装函数内取得，才能测量每次调用。`perf_counter()` 适合测量时间间隔；秒乘 1000 转为毫秒。`finally` 确保发生异常也记录时间，但不吞掉异常。原题 `slow(11, 22, 33)` 应返回 7986，具体耗时随机器和调度变化，不应写死。

E23 前后日志与两种装饰语法

题意改写 | 原 8.4 后两问：打印 begin/end，并同时支持 `@log` 与 `@log('execute')`。

```
from functools import wraps

def log(arg=None):
    def decorate(fn, text):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            print('begin call', text, fn.__name__)
            try:
                return fn(*args, **kwargs)
            finally:
                print('end call', text, fn.__name__)
        return wrapper
    if callable(arg):
        return decorate(arg, '')
    return lambda fn: decorate(fn, '' if arg is None else arg)

@log
def one():
    return 1

@log('execute')
def two():
    return 2

assert one() == 1 and two() == 2
```

详解：裸装饰时传入的是函数，直接生成包装；带字符串时第一次调用只接收日志标签并返回装饰器，第二次才接收函数。前后日志在真正调用时打印。把 end 放入 `finally`，即使被装饰函数抛错也能保留结束日志。

S05 固定进制与冻结循环变量

```
from functools import partial
int2 = partial(int, base=2)
assert int2('1010') == 10
functions = [lambda x=i: x * x for i in range(1, 4)]
assert [f() for f in functions] == [1, 4, 9]
```

解析：偏函数固定二进制解析；默认参数 `x=i` 在每次 lambda 创建时求值。若写成 `lambda: i * i`，三个函数之后通常都读取循环结束时的同一个 `i`。

> **验收** 能区分“返回函数”与“返回调用结果”，并解释装饰器为什么需要三层函数或参数分派。

06 模块与面向对象

来源章节：9 模块、9.1—9.2；10 面向对象编程、10.1—10.5。

K14 模块、包与安装环境

- 一个 `.py` 文件通常是一个模块；包用层级组织模块，常规包用 `__init__.py` 初始化。模块名避免与 `json.py`、`email.py` 等标准库同名。
- `import module` 后用 `module.name`；`from module import name` 直接引入该名字。模块搜索路径见 `sys.path`，不要把临时修改路径当成可靠的项目安装方案。
- `if __name__ == '__main__':` 中的入口代码只在直接运行时执行，导入时不执行这部分。
- 安装包优先用 `python3 -m pip install package`，保证 pip 属于选中的解释器。发行包名与导入名可不同：安装 Pillow，导入 PIL。

S06 可导入的脚本

```
def greet(name='world'):
    return f'Hello, {name}!'

if __name__ == '__main__':
    import sys
    print(greet(sys.argv[1] if len(sys.argv) > 1 else 'world'))
```

解析： `sys.argv[0]` 是脚本名，额外参数从索引 1 开始。导入这个模块后仍可调用 `greet()`，不会自动运行打印分支。

K15 类、实例、封装与多态

- 类是创建实例的模板，实例保存自身状态；`__init__` 初始化实例，实例方法的第一个参数按惯例命名 `self`。
- `self.name` 是实例属性；类体中定义的 `count` 等是类属性。不要把需要每个实例独立拥有的可变列表放成共享类属性。
- `__name` 触发名字改写，用来减少意外访问；不是安全边界。`_name` 主要表示内部使用约定；`__name__` 通常是协议名称。
- 子类可重写方法；调用同一接口得到不同对象的行为就是多态。鸭子类型关注对象能完成什么，不强制共同继承某个类。
- `type()` 看具体类型，`isinstance()` 考虑继承；`dir()` 列名字；`getattr(obj, name, default)` 取属性；`setattr()` 设置；`hasattr()` 内部会尝试取属性。

E24 隐藏并校验 gender

题意改写 | 原 10.2：通过 getter/setter 访问性别，拒绝题目范围之外的值。

```
class Student:
    def __init__(self, name, gender):
        self.name = name
        self.set_gender(gender)

    def get_gender(self):
        return self.__gender

    def set_gender(self, gender):
        if gender not in ('male', 'female'):
            raise ValueError('invalid gender')
        self.__gender = gender

bart = Student('Bart', 'male')
bart.set_gender('female')
assert bart.get_gender() == 'female'
```

详解：构造时也调用 setter，确保初始值与后续修改遵循同一校验规则。先验证、后赋值，错误输入不会破坏原有状态。取值集合沿用原练习，不是对实际业务字段的完整建模。

E25 类属性统计实例创建次数

题意改写 | 原 10.5：每构造一个 Student，类计数加一。

```
class Student:
    count = 0
    def __init__(self, name):
        self.name = name
        Student.count += 1

bart, lisa = Student('Bart'), Student('Lisa')
assert Student.count == 2
```

详解：明确写 `Student.count` 修改共享的类属性。写 `self.count += 1` 时，读取可能来自类，但赋值会在实例上生成同名属性，不能实现预期的共享统计。这里统计累计创建次数，不是当前仍存活对象数。

S07 继承、鸭子类型与对象信息

```
class Animal:
    def run(self):
        return 'animal'

class Dog(Animal):
    def run(self):
        return 'dog'

def run_twice(obj):
    return obj.run(), obj.run()

d = Dog()
assert isinstance(d, Animal)
assert run_twice(d) == ('dog', 'dog')
assert getattr(d, 'missing', None) is None
```

解析：`run_twice` 不要求传入 Animal，只要求对象有可调用的 `run`。`type(d) is Animal` 为假，但 `isinstance(d, Animal)` 为真，因为后者考虑继承关系。

验收 能解释实例属性如何遮蔽类属性，并区分封装约定、名字改写和真正的访问控制。

07 面向对象高级特性

来源章节：11 面向对象高级编程、11.1—11.6。

K16 限制属性与 property

- `__slots__` 以名字集合限制常规实例可动态新增的属性，并可能节约内存；继承规则会影响它是否仍有限制，不应当作权限机制。
- `@property` 把方法包装成属性读取接口；`@name.setter` 处理赋值，可在赋值前校验；没有 setter 的属性在常规赋值时只读。
- 多重继承按 MRO 查找方法。MixIn 适合提供小范围附加行为；协作继承中用 `super()`，避免硬编码父类引发重复初始化。

E26 Screen 的宽、高与只读面积

题意改写 | 原 11.2：设置 width、height，并只读地得到 resolution。

```
class Screen:
    @property
    def width(self):
        return self._width

    @width.setter
    def width(self, value):
        if type(value) is not int or value <= 0:
            raise ValueError('width must be a positive integer')
        self._width = value

    @property
    def height(self):
        return self._height

    @height.setter
    def height(self, value):
        if type(value) is not int or value <= 0:
            raise ValueError('height must be a positive integer')
        self._height = value

    @property
    def resolution(self):
        return self.width * self.height

s = Screen()
s.width, s.height = 1024, 768
assert s.resolution == 786432
```

详解：对外使用属性语法，内部由方法检查数据。`resolution` 每次根据当前宽高计算，不额外存储，避免旧缓存。`type(value) is int` 刻意排除布尔值，因为 `bool` 是 `int` 的子类。需先设置宽高，再读取；直接给 `s.resolution` 赋值会抛出 `AttributeError`。

K17 定制协议与枚举

- `__str__` 面向用户展示，`__repr__` 面向调试；`__len__` 支持 `len(obj)`。
- `__iter__` 返回迭代器；迭代器用 `__next__` 产出下一项，结束时抛出 `StopIteration`。
- `__getitem__` 支持索引，也可能收到 `slice`；要明确负索引、步长和边界的支持范围。`__setitem__`、`__delitem__` 对应修改与删除。
- `__getattr__` 在常规查找失败后才调用，无法处理的名字应抛出 `AttributeError`。`__call__` 允许把实例像函数一样调用，`callable()` 检查可调用性。
- `Enum` 建立明确的命名成员；`.name` 取名字、`.value` 取值；`@unique` 拒绝重复值。

E27 枚举表示性别

题意改写 | 原 11.5：把 Student 的性别从字符串改为 Gender 成员。

```
from enum import Enum, unique

@unique
class Gender(Enum):
    Male = 0
    Female = 1

class Student:
    def __init__(self, name, gender):
        if not isinstance(gender, Gender):
            raise TypeError('expected Gender')
        self.name, self.gender = name, gender

bart = Student('Bart', Gender.Male)
assert bart.gender is Gender.Male
assert bart.gender.value == 0
```

详解：枚举成员具有类型身份，避免字符串拼写失误。`Gender.Male` 不是普通整数 0；从值构造用 `Gender(0)`，从成员名字查找用 `Gender['Male']`。

S08 动态链式路径

原 11.4 扩展示例：补全 `Chain().users('michael').repos`。

```
from urllib.parse import quote

class Chain:
    def __init__(self, path=''):
        self._path = path

    def __getattr__(self, name):
        if name.startswith('_'):
            raise AttributeError(name)
        return Chain(f'{self._path}/{name}')

    def __call__(self, value):
        return Chain(f'{self._path}/{quote(str(value), safe="")}')

    def __str__(self):
        return self._path

    __repr__ = __str__

assert str(Chain().users('michael').repos) == '/users/michael/repos'
```

详解：不存在的属性构建一个新路径对象；调用对象时把参数作为一个经过 URL 编码的路径段追加。每一步都返回新对象，因此可持续链式调用。它只构造路径，未请求网络。

K18 type、元类与 ORM 思路

`type(name, bases, namespace)` 可动态创建类；常规类本身通常也是 `type` 的实例。元类通过创建类的过程检查或改写类定义。原教程 ORM 示例把类属性中的 Field 收集为映射，再在 `save()` 中生成 SQL 和参数；这展示元类用途，不等于已经有事务、迁移、类型适配和完整查询功能的 ORM。

S09 动态创建类与注入方法

```
def hello(self, name='world'):
    return f'Hello, {name}!'

Hello = type('Hello', (object,), {'hello': hello})
assert Hello().hello() == 'Hello, world!'

class ListMetaclass(type):
    def __new__(mcls, name, bases, attrs):
        attrs['add'] = lambda self, value: self.append(value)
        return super().__new__(mcls, name, bases, attrs)

class MyList(list, metaclass=ListMetaclass):
    pass

items = MyList()
items.add(1)
assert items == [1]
```

详解：先创建类，再由类创建实例。`ListMetaclass.__new__` 接收正在定义的类名、基类元组和属性字典，注入 `add` 后交回标准建类逻辑。普通应用应先熟练使用类与组合，再研究元类。

> **验收** 能对应“写法 → 协议方法”，能说明 ORM 是如何从对象字段映射到 SQL 列的。

08 异常、调试与测试

来源章节：12 错误、调试和测试、12.1—12.4。

K19 异常传播与调试方法

- `try` 放可能失败的操作；`except` 处理选定异常；`else` 只在 try 正常结束时运行；`finally` 无论成功失败都运行。
- 异常类型从具体到一般捕获；用 `raise` 主动报告问题，裸 `raise` 重新抛出当前异常。异常会沿调用链传播，直到被处理或程序退出。
- 先读 traceback 最后的异常类型和消息，再沿调用栈定位源头。`logging.exception()` 在异常处理块内保留堆栈。
- `assert` 用于开发期不变量检查，在优化模式可能被关闭；输入校验不能只靠 assert。`breakpoint()`/pdb 用于暂停、单步、看变量。

易错 不要在 finally 中 return，它可能覆盖原有返回值或异常。不要用宽泛的 except 后直接 pass，否则会隐去失败原因。

E28 修复带小数的求和程序

题意改写 | 原 12.1：原函数只用 `int()`，处理包含 7.6 的加法字符串时失败。

```
from functools import reduce

def str2num(s):
    s = s.strip()
    try:
        return int(s)
    except ValueError:
        return float(s)

def calc(exp):
    values = map(str2num, exp.split('+'))
    return reduce(lambda acc, x: acc + x, values)

assert calc('100 + 200 + 345') == 645
assert abs(calc('99 + 88 + 7.6') - 194.6) < 1e-10
```

详解：错误源是 `int('7.6')`，不是 reduce。先尝试整数，失败后再转浮点数，保留原整数表达式结果类型。第二次转换仍失败时让异常继续传播，不静默把非法内容当 0。这里只实现以加号分隔的数值求和，不是完整算式解析器。

K20 单元测试与文档测试

`unittest.TestCase` 的 `assertEqual` 比较结果，`assertRaises` 检查异常；`setUp`/`tearDown` 准备和清理测试环境。好的测试覆盖普通值、边界值、非法值；不只验证一次正常运行。doctest 从文档字符串中抽取交互式示例并比对预期输出。

E29 修复 `Student.get_grade`

题意改写 | 原 12.3：让分数边界与非法分数的全部测试通过。

```
class Student:
    def __init__(self, name, score):
        self.name, self.score = name, score

    def get_grade(self):
        if not 0 <= self.score <= 100:
            raise ValueError('score out of range')
        if self.score >= 80:
            return 'A'
        if self.score >= 60:
            return 'B'
        return 'C'
```

详解：原程序先判断 60，80 分以上也立刻返回 B，因此高分分支必须前置。区间为 A：[80,100]，B：[60,80)，C：[0,60)；先检查 -1 与 101 等越界输入。原测试在调用 `get_grade()` 时期待异常，所以保持构造函数允许暂存该数值，把范围校验放在方法里。
验收数据：0、59→C；60、79→B；80、100→A；-1、101→ 抛出 `ValueError`。

E30 补全阶乘 doctest

题意改写 | 原 12.4：写出 fact(10)、fact(-1) 的预期行为并执行文档测试。

```
def fact(n):
    '''
    >>> fact(1)
    1
    >>> fact(10)
    3628800
    >>> fact(-1)
    Traceback (most recent call last):
        ...
    ValueError
    '''
    if n < 1:
        raise ValueError()
    if n == 1:
        return 1
    return n * fact(n - 1)

if __name__ == '__main__':
    import doctest
    doctest.testmod(verbose=True)
```

详解：成功示例写实际返回值；失败示例写 traceback 形式及最后的异常类型。此函数沿用原题 `n>=1` 的定义域；数学上的 `0! = 1` 未纳入接口，不能在不改函数的情况下给 `fact(0)` 填 1。

> **验收** 能指出一个错误是语法、类型、数值范围还是分支逻辑问题，并给出至少一组能暴露它的测试。

09 文件、目录与序列化

来源章节：13 IO 编程、13.1—13.4。

K21 资源管理与读写方式

- 同步 IO 会让当前执行流程等待操作完成；文件打开后应及时关闭。`with open(...) as f` 保证离开上下文时关闭。
- `r` 读文本，`w` 创建或截断写入，`a` 追加，`rb/wb` 读写字节。文本建议显式设置 `encoding='utf-8'`；二进制模式不接受 encoding。
- `read()` 可一次读完整文件，内存不足时分块读或逐行迭代；`readline()` 取一行；`write()` 返回写入字符数或字节数。
- `StringIO` 是内存文本流，`BytesIO` 是内存字节流；`getvalue()` 取全部缓冲内容，当前位置影响 `read()`。
- `os.path.join` 组合路径，`os.listdir` 列目录，`os.walk` 递归遍历；`os.rename`、`os.remove` 会改变文件系统，应明确目标路径。

E31 读取本地文本

题意改写 | 原 13.1：读本地文本为字符串并打印；原文示例路径依赖系统。

```
from pathlib import Path

fpath = Path('sample.txt')
# Preparation for this self-contained example:
fpath.write_text('你好，Python！\n', encoding='utf-8')
with open(fpath, 'r', encoding='utf-8') as f:
    s = f.read()
print(s, end='')
```

详解：这里先在练习目录创建 sample.txt，使示例能独立运行；读取已有文件时删去准备行，换成实际路径，避免覆盖已有内容。原 `/etc/timezone` 并非每个系统都存在；路径不存在会抛出 `FileNotFoundError`。本例输出一行“你好，Python！”。

E32 输出类似 dir -l 的目录明细

题意改写 | 原 13.3 第一问：列出当前目录各项类型、权限、大小、修改时间和名字。

```
import os
import stat
from datetime import datetime

def list_long(path='.'):
    with os.scandir(path) as entries:
        for entry in sorted(entries, key=lambda e: e.name):
            info = entry.stat(follow_symlinks=False)
            mode = stat.filemode(info.st_mode)
            modified = datetime.fromtimestamp(info.st_mtime)
            stamp = modified.strftime('%Y-%m-%d %H:%M')
            print(f'{mode} {info.st_size:>10} {stamp} {entry.name}')

# list_long('.')
```

详解： `scandir` 提供目录项与状态；`stat.filemode` 同时显示类型和权限，例如目录首字符为 d。`follow_symlinks=False` 读取链接自身信息。这里实现长列表的主要字段，不保证逐字复刻某个操作系统的命令输出，目录大小也不是递归总大小。

E33 递归按文件名片段搜索

题意改写 | 原 13.3 第二问：打印当前目录和子目录里文件名包含指定文本的相对路径。

```
import os

def find_files(keyword, root='.'):
    for folder, dirs, files in os.walk(root):
        dirs.sort()
        for name in sorted(files):
            if keyword in name:
                full = os.path.join(folder, name)
                print(os.path.relpath(full, root))

# find_files('test')
```

详解： 只在 `files` 中搜索，不把同名目录算作文件；用 `relpath` 去掉根目录前缀。此实现区分大小写；若要忽略大小写，应同时比较 `keyword.casefold()` 与 `name.casefold()`。`os.walk` 默认不递归进入目录符号链接，避免常见循环路径。

K22 JSON 与 pickle

JSON 常见对应：对象 →dict、数组 →list、字符串 →str、null→None、布尔值 →bool。`dumps/loads` 在字符串之间转换；`dump/load` 操作文件。自定义类需要 `default` 或显式转换为普通结构；反序列化可用 `object_hook`。

> **易错** pickle 能保存更多 Python 对象，但其加载过程可执行构造逻辑。只对可信来源使用 pickle；跨语言交换优先选择受约束的 JSON 等格式。

E34 `ensure_ascii` 的区别

题意改写 | 原 13.4：比较中文序列化时的两种输出。

```
import json
obj = {'name': '小明', 'age': 20}
a = json.dumps(obj, ensure_ascii=True)
b = json.dumps(obj, ensure_ascii=False)
print(a)
print(b)
assert json.loads(a) == json.loads(b) == obj
```

详解：第一种把中文表示为 `\u5c0f\u660e`；第二种直接显示“小明”。两者解析后的数据相同；`ensure_ascii` 控制 JSON 文本的转义，不负责把文本写成 UTF-8 字节。写文件时仍要设置 encoding。

S10 内存流与对象编码

```
from io import StringIO, BytesIO
import json

text = StringIO()
text.write('hello')
assert text.getvalue() == 'hello'
binary = BytesIO('中文'.encode('utf-8'))
assert binary.read().decode('utf-8') == '中文'

class Student:
    def __init__(self, name):
        self.name = name

s = json.dumps(Student('Alice'), default=lambda x: vars(x))
assert json.loads(s) == {'name': 'Alice'}
```

解析：`StringIO` 接收字符串，`BytesIO` 接收字节；`default` 只为 JSON 原本不支持的对象提供转换，不能把任意对象的内部状态都不加选择地公开。

> **验收** 会选读写模式、解释截断写入的含义，并能区分 JSON 字符串、字节和解析后的 Python 对象。

10 多进程、多线程与分布式任务

来源章节：14 进程和线程、14.1—14.5。

K23 并发模型比较

- 进程通常有独立地址空间，通信可用 Queue、Pipe 等；线程共享进程内存，共享状态需要同步。`start()` 启动任务，`join()` 等待结束，直接调用 `run()` 不会新建线程或进程。
- `multiprocessing.Process` 创建进程，Pool 管理进程池；跨平台脚本入口放入 `if __name__ == '__main__':`，工作函数应能被子进程导入。`os.fork()` 只适用于支持它的平台，不是可移植入口。
- `subprocess.run()` 调用外部程序，可设 `check=True` 检查退出码、`capture_output=True` 收集输出。
- 线程的读改写可能发生竞争。`with lock:` 保证临界区退出时释放锁；锁住最小必要范围，避免死锁与无谓串行化。
- `threading.local()` 为每个线程提供独立属性；线程池会复用线程，处理新任务时仍需正确设置上下文。

重点 在常见的带 GIL 的 CPython 构建中，纯 Python 的 CPU 密集任务通常优先考虑多进程或释放 GIL 的扩展；IO 密集任务可考虑线程或异步。Python 也存在可选的自由线程构建，不能把“任何 Python 都只能一个线程执行”当作永恒定律。并发数量增加会带来调度、内存和连接成本。

易错 普通 Python 线程抛出未处理异常，一般结束该线程；不是必然让整个进程崩溃。原生扩展导致的进程级崩溃属于另一类问题。子进程崩溃的隔离性也不意味着共享外部资源绝对不受影响。

S11 进程池计算与线程锁

配套补充丨分别保存运行；不在交互单元中直接测试进程启动。

```
from multiprocessing import get_context

def square(x):
    return x * x

if __name__ == '__main__':
    with get_context('spawn').Pool(2) as pool:
        print(pool.map(square, [1, 2, 3]))
```

详解：显式使用 spawn 演示可移植的入口组织；父进程向两个工作进程分发任务，`map` 按输入顺序收集结果 `[1,4,9]`。工作进程不共享父进程中普通列表的后续改动。

```
import threading

balance = 0
lock = threading.Lock()

def add_many():
    global balance
    for _ in range(1000):
        with lock:
            balance += 1

threads = [threading.Thread(target=add_many) for _ in range(2)]
for t in threads:
    t.start()
for t in threads:
    t.join()
assert balance == 2000
```

详解：两个线程共享 balance；锁让每次完整更新保持互斥。`join()` 后才读取最终结果。不要依靠某个解释器版本中短片段“刚好没有出错”来证明不需要同步。

K24 分布式进程与任务队列

`multiprocessing.managers.BaseManager` 可把队列通过代理提供给其他进程甚至另一台机器。服务端先注册接口并启动；工作端以相同名字注册后连接，再从代理取任务、交回结果。需要明确超时、失败重试和任务标识；`authkey` 用于鉴别，不等于对传输数据做了完整加密。

S12 最小分布式队列的两端

原 14.5 示例整理丨先运行服务端；再在另一个终端运行工作端。仅示范本机通信。

```
# manager_server.py
import queue
from multiprocessing.managers import BaseManager

tasks, results = queue.Queue(), queue.Queue()

def get_tasks():
    return tasks

def get_results():
    return results

class Manager(BaseManager):
    pass

Manager.register('get_tasks', callable=get_tasks)
Manager.register('get_results', callable=get_results)

if __name__ == '__main__':
    tasks.put(12)
    manager = Manager(address=('127.0.0.1', 5000), authkey=b'demo')
    manager.get_server().serve_forever()
```

```
# manager_worker.py
from multiprocessing.managers import BaseManager

class Manager(BaseManager):
    pass

Manager.register('get_tasks')
Manager.register('get_results')
manager = Manager(address=('127.0.0.1', 5000), authkey=b'demo')
manager.connect()
n = manager.get_tasks().get(timeout=5)
manager.get_results().put((n, n * n))
print('result submitted:', n * n)
```

详解：服务端持有真实队列；客户端拿到的是代理。任务 12 被转换成 `(12,144)` 放入结果队列。服务端控制台不会自动显示结果，若需要汇总应另加 `get_results().get(timeout=...)` 的协调端；示例只演示注册、连接、取任务和回传四步。

> **验收** 能根据“CPU/IO、是否共享状态、是否需要跨机器”选模型，并说明锁和队列各解决什么问题。

11 正则表达式

来源章节：15 正则表达式。

K25 必记语法与匹配范围

- `\d` 数字、`\w` 单词字符、`\s` 空白；`.` 通常匹配换行以外的一个字符；`[]` 字符集合；`()` 分组；`|` 候选。
- `*` 零或多次，`+` 一或多次，`?` 零或一次，`{m,n}` 次数区间；量词后加 `?` 可改为非贪婪。
- `^/$` 表示边界，但 `$` 有末尾换行等语义；严格验证整个字符串时优先 `fullmatch()`。`match()` 从开头匹配，`search()` 在任意位置寻找。
- 原始字符串 `r'...'` 避免 Python 再处理反斜杠；`re.compile()` 便于复用；分组结果用 `group()`，拆分可用 `re.split()`。

E35 按题目规则验证邮箱

题意改写 | 原 15 第一问：接受常规示例，拒绝原题特别指定的带连字符本地部分。

```
import re

PATTERN = re.compile(
    r'[A-Za-z0-9_]+(?:\.[A-Za-z0-9_]+)*'
    r'@(?:[A-Za-z0-9](?:[A-Za-z0-9-]*[A-Za-z0-9])?\.)+'
    r'[A-Za-z]{2,}'
)

def is_valid_email(addr):
    return PATTERN.fullmatch(addr) is not None

assert is_valid_email('someone@gmail.com')
assert is_valid_email('bill.gates@microsoft.com')
assert not is_valid_email('bob#example.com')
assert not is_valid_email('mr-bob@example.com')
```

详解：本地部分允许字母、数字、下划线和中间单点；域名由点分隔，顶级域至少两个字母；完整匹配防止后面夹带其他字符。为符合原题测试，刻意不允许本地部分出现 `-`；真实邮箱标准允许很多这里未覆盖的情况，因此这个答案是练习规则，不是完整邮箱标准验证器。

E36 提取显示名或邮箱用户名

题意改写 | 原 15 第二问：`<Tom Paris> tom@voyager.org` 取显示名；无显示名时取 @ 前的名字。

```
import re

def name_of_email(addr):
    match = re.fullmatch(r'\s*<([^<>]+)>\s*([^\s@]+@[^\s@]+)\s*', addr)
    if match:
        return match.group(1).strip()
    match = re.fullmatch(r'\s*([^\s@]+)@[^\s@]+\s*', addr)
    if match:
        return match.group(1)
    raise ValueError('invalid exercise format')

assert name_of_email('<Tom Paris> tom@voyager.org') == 'Tom Paris'
assert name_of_email('bob@example.com') == 'bob'
```

详解：优先匹配带尖括号的显示名格式；无显示名时再捕获用户名。这里沿用题目“名字在尖括号中”的写法；真实邮件头的常见结构是 `Name <addr>`，应交给 `email.utils.parseaddr` 等邮件解析工具。

验收 能解释 `+` 与 `*` 的区别、分组与字符集合的区别，并指出题目规则与实际协议标准的边界。

12 常用内建模块：时间与数据组织

来源章节：16 常用内建模块、16.1 datetime、16.2 collections、16.3 argparse。

K26 datetime 与时区

`datetime` 表示日期时间；`timedelta` 表示时间差；`timezone` 表示固定偏移时区。`strptime` 将字符串解析为时间，`strftime` 按格式输出；timestamp 表示相对 Unix 纪元的秒数，同一时刻不因显示时区而改变。

重点 naive 时间不带时区，aware 时间带时区。`replace(tzinfo=...)` 给现有钟面值附加解释；`astimezone(...)` 保持同一时刻并改变显示。不能把“附加时区”与“转换时区”混为一谈。地区夏令时需要 `zoneinfo` 等地区时区规则，而不是固定偏移。

E37 带固定时区的时间转 timestamp

题意改写 | 原 16.1：输入日期字符串及 `UTC+7:00`、`UTC-09:00` 等偏移，返回时间戳。

```
import re
from datetime import datetime, timezone, timedelta

def to_timestamp(dt_str, tz_str):
    m = re.fullmatch(r'UTC([+-])(\d{1,2}):(\d{2})', tz_str)
    if not m:
        raise ValueError('invalid timezone')
    hour, minute = int(m[2]), int(m[3])
    if hour > 23 or minute > 59:
        raise ValueError('invalid offset')
    offset = timedelta(hours=hour, minutes=minute)
    if m[1] == '-':
        offset = -offset
    local = datetime.strptime(dt_str, '%Y-%m-%d %H:%M:%S')
    return local.replace(tzinfo=timezone(offset)).timestamp()

assert to_timestamp('2015-6-1 08:10:30', 'UTC+7:00') == 1433121030.0
assert to_timestamp('2015-5-31 16:10:30', 'UTC-09:00') == 1433121030.0
```

详解：先解析偏移符号与时分，给没有时区的钟面值附上指定偏移，再由标准库换算为统一时间戳。负号应用于整个偏移，不是只应用于小时。两个测试描述同一时刻，所以返回值相同，且不依赖运行电脑的本地时区。

K27 collections 选择表

- `namedtuple`：有名字的轻量元组，适合坐标等固定记录。
- `deque`：两端增删高效；`appendleft/popleft` 适合队列。
- `defaultdict(factory)`：访问缺失键时创建默认值，`get()` 不会触发相同的创建行为。
- `OrderedDict`：保留显式顺序操作，例如移动到末尾；普通 dict 已能保留插入顺序。

- `ChainMap`：按前后多个映射依次查找，适合命令行配置覆盖环境和默认配置；写入通常发生在第一层。
- `Counter`：统计可哈希元素出现次数，`most_common()` 取高频项。

S13 计数与配置优先级

```
from collections import Counter, deque, ChainMap, namedtuple

Point = namedtuple('Point', ['x', 'y'])
assert Point(2, 3).x == 2
q = deque(['a', 'b'])
assert q.popleft() == 'a'
assert Counter('banana').most_common(1) == [('a', 3)]
config = ChainMap({'color': 'blue'}, {'color': 'gray', 'size': 10})
assert config['color'] == 'blue' and config['size'] == 10
```

解析：为具体操作选合适的数据结构，比先写大量手工循环更清楚；ChainMap 的覆盖顺序由映射在参数中的先后位置决定。

K28 argparse 命令行解析

`ArgumentParser` 管理参数声明、类型转换和帮助信息；`add_argument` 声明参数，`parse_args` 得到 Namespace。位置参数通常必填；`--name` 是可选参数名，是否必须由 `required` 等配置决定；布尔开关通常用 `action='store_true'`，不用 `type=bool`。

```
import argparse
parser = argparse.ArgumentParser(description='Repeat greeting')
parser.add_argument('name')
parser.add_argument('--count', type=int, default=1)
parser.add_argument('--loud', action='store_true')
args = parser.parse_args(['Alice', '--count', '2', '--loud'])
assert args.name == 'Alice' and args.count == 2 and args.loud
```

解析：提供列表便于复现；真实脚本用 `parse_args()` 从命令行读取。字符串 `'False'` 本身也是真值，因此 `type=bool` 不适合解析开关。

13 常用内建模块：二进制、摘要与资源管理

来源章节：16.4 base64、16.5 struct、16.6 hashlib、16.7 hmac、16.8 itertools、16.9 contextlib。

K29 Base64 与 struct

Base64 把二进制编码为可打印字符，常见编码长度按 4 个字符成组，= 用于填充；它可逆，不是加密。`urlsafe_b64encode` 使用适合 URL 的字符。`struct.pack/unpack` 在 Python 数值与固定二进制布局之间转换；> 大端，< 小端，`I` 常表示无符号 4 字节整数；用 `calcsize()` 明确布局长度。

E38 补齐 Base64 的等号

题意改写 | 原 16.4：支持标准 Base64 文本省略末尾填充。

```
import base64

def safe_base64_decode(s):
    data = s.encode('ascii') if isinstance(s, str) else s
    data += b'=' * (-len(data) % 4)
    return base64.b64decode(data, validate=True)

assert safe_base64_decode('YWJjZA==') == b'abcd'
assert safe_base64_decode('YWJjZA') == b'abcd'
```

详解：`-len(data) % 4` 得到补到 4 的倍数所需的 0—3 个等号，已对齐时补 0 个。`validate=True` 拒绝非法字符；它只修复题目规定的“缺末尾填充”，不能把任意损坏数据恢复出来。

S14 读取 BMP 文件头的基本信息

原 16.5 讲解示例整理 | 只读取未压缩格式常见文件头字段，不解码图像像素。

```
import struct

def bmp_info(data):
    if len(data) < 30:
        raise ValueError('header too short')
    signature, size, r1, r2, offset, header_size = struct.unpack(
        '<2sIHHII', data[:18])
    if signature != b'BM' or header_size < 40:
        raise ValueError('expected Windows BMP header')
    width, height, planes, bits = struct.unpack('<iiHH', data[18:30])
    return {'width': width, 'height': abs(height), 'color': bits}

header = struct.pack('<2sIHHIIiiHH',
                     b'BM', 1000, 0, 0, 54, 40, 640, 480, 1, 24)
assert bmp_info(header) == {'width': 640, 'height': 480, 'color': 24}
```

详解：文件头采用小端字节序；位图宽高字段是有符号整数，负高度可表示从上到下存储，所以输出高度取绝对值。检查长度和签名后再解包；这仍不是对 BMP 全部合法性的验证。

K30 哈希、加盐与 HMAC

`hashlib` 的 `update(bytes)` 累计输入，`digest()` 返回摘要字节，`hexdigest()` 返回十六进制文本。相同消息产生相同摘要；摘要不是可逆加密。只有未经可信校验的“文件 + 摘要”一起被替换时，摘要本身不能证明消息来源。

HMAC 用密钥和消息构造认证码；`hmac.compare_digest()` 适合比较认证码。盐一般可以与摘要一起保存，不需要秘密；独立随机盐可减少相同密码产生相同记录和预计算复用。

> **易错** 以下 E39—E41 为复现教程练习，保留 MD5/HMAC-MD5 的题设接口。真实密码存储不应直接使用快速 MD5、普通 SHA 摘要或单轮 HMAC；应使用专用、可调成本的密码派生方案。此处是对教程旧示例的用途限定，不把其“加盐后安全”表述当作现代密码存储结论。

E39 MD5 与固定摘要登录验证

题意改写 | 原 16.6 第一练习：计算密码摘要，和题目数据库比较。

```
import hashlib
import hmac

def calc_md5(password):
    return hashlib.md5(password.encode('utf-8')).hexdigest()

db = {
    'michael': 'e10adc3949ba59abbe56e057f20f883e',
    'bob': '878ef96e86145580c38c87f0410ad153',
    'alice': '99b1c2188db85afee403b1536010c2c9'
}

def login(user, password):
    stored = db.get(user)
    return stored is not None and hmac.compare_digest(
        stored, calc_md5(password))

assert login('michael', '123456')
assert login('bob', 'abc999')
assert login('alice', 'alice2008')
assert not login('alice', 'Alice2008')
assert not login('missing', '123456')
```

详解：摘要函数接收字节，所以先 UTF-8 编码。计算结果是十六进制字符串，与数据库类型一致；密码大小写不同会得到不同摘要。用 `db.get` 使不存在的用户直接返回 False。固定快速摘要容易被字典枚举，这个答案仅完成原题机制。

E40 用户盐的注册与登录

题意改写 | 原 16.6 第二练习：同时补全“用户名拼接固定盐”和“每用户独立盐”两种思路。

```
import hashlib
import hmac
import secrets

def md5_text(s):
    return hashlib.md5(s.encode('utf-8')).hexdigest()

fixed_db = {}
def register_fixed(username, password):
    fixed_db[username] = md5_text(password + username + 'the-Salt')

def login_fixed(username, password):
    stored = fixed_db.get(username)
    candidate = md5_text(password + username + 'the-Salt')
    return stored is not None and hmac.compare_digest(stored, candidate)

class User:
    def __init__(self, username, password):
        self.username = username
        self.salt = secrets.token_hex(16)
        self.password = get_md5(self, password)

def get_md5(user, pws):
    return md5_text(pws + user.salt)

db = {'michael': User('michael', '123456'),
      'bob': User('bob', 'abc999'), 'alice': User('alice', 'alice2008')}
def login(username, password):
    user = db.get(username)
    return user is not None and hmac.compare_digest(
        user.password, get_md5(user, password))

register_fixed('bob', 'abc999')
assert login_fixed('bob', 'abc999')
assert login('michael', '123456')
assert not login('michael', '1234567')
```

详解：注册与验证必须拼接完全相同的内容；随机盐只在创建用户时生成一次，登录时读取已保存的盐，不能重抽。原题同时出现单参数、双参数的 `get_md5` 名称，直接照拼会产生签名冲突；这里用 `md5_text` 与 `get_md5(user,pws)` 分离职责。盐生成改用 `secrets`，但摘要机制仍保留为旧练习。

E41 用 HMAC 验证口令

题意改写 | 原 16.7：每用户保存 key，通过标准 HMAC 重新计算后比较。

```
import hmac
import secrets

def hmac_md5(key, s):
    return hmac.new(key.encode('utf-8'), s.encode('utf-8'),
                    'md5').hexdigest()

class User:
    def __init__(self, username, password):
        self.username = username
        self.key = secrets.token_hex(16)
        self.password = hmac_md5(self.key, password)

db = {'michael': User('michael', '123456'),
      'bob': User('bob', 'abc999'), 'alice': User('alice', 'alice2008')}

def login(username, password):
    user = db.get(username)
    return user is not None and hmac.compare_digest(
        user.password, hmac_md5(user.key, password))

assert login('michael', '123456')
assert not login('michael', 'wrong')
```

详解：`hmac.new` 的参数顺序是 key、消息、摘要算法；key 与消息都需字节。登录使用同一用户已保存的 key。该练习帮助区分普通摘要和带密钥摘要，不构成现代密码存储系统。

K31 itertools 与 contextlib

- `count(start, step)` 无限计数，`cycle(xs)` 无限循环，`repeat(x,n)` 重复；使用无限迭代器前先确定终止条件。
- `islice` 限定提取范围，`chain` 连接输入，`takewhile` 保留开头连续满足条件的部分。
- `groupby` 只把相邻且键相同的元素归为一组；全局分组通常先按同一键排序。组迭代器共享输入，应及时消费。
- `@contextmanager` 将单次 yield 的生成器变成上下文管理器：yield 前准备资源，yield 的值交给 as，finally 中清理；`closing` 关闭只有 close 接口的对象。

E42 itertools 近似计算圆周率

题意改写 | 原 16.8：计算莱布尼茨级数前 N 项和。

```
import itertools
import math

def pi(N):
    if type(N) is not int or N <= 0:
        raise ValueError('N must be a positive integer')
    odds = itertools.islice(itertools.count(1, 2), N)
    return math.fsum(4 / x if i % 2 == 0 else -4 / x
                     for i, x in enumerate(odds))

assert 3.04 < pi(10) < 3.05
assert 3.13 < pi(100) < 3.14
assert 3.140 < pi(1000) < 3.141
assert 3.1414 < pi(10000) < 3.1415
```

详解：奇数序列是 1、3、5……；序号从 0 开始，偶数序号取正，奇数取负。公式为 $\pi \approx 4\sum_{k=0}^{N-1}(-1)^k/(2k+1)$。`islice` 截断无限序列；`fsum` 减少累计求和误差。该级数收敛很慢，增加 N 并不意味着迅速得到高精度。

S15 自定义上下文管理器

```
from contextlib import contextmanager

@contextmanager
def managed_list():
    resource = []
    try:
        yield resource
    finally:
        resource.clear()

with managed_list() as values:
    values.append(1)
    assert values == [1]
assert values == []
```

详解：yield 把资源交给 with 块；正常退出或抛异常都会执行 finally。上下文管理器的生成器必须只 yield 一次，不能把它当成普通多项生成器。

14 常用内建模块：HTTP、XML、HTML 与 venv

来源章节：16.10 urllib、16.11 XML、16.12 HTMLParser、16.13 venv。

K32 HTTP 客户端与解析器

`urllib.request` 用于 HTTP 请求；`urlencode` 编码查询参数；响应体首先是字节。先确认 HTTP 状态、内容类型和字符集，再交给 JSON/XML/HTML 解析器。

XML：DOM 构建整棵树，随机访问方便；SAX/Expat 触发开始标签、字符数据、结束标签事件，适合流式处理。字符数据回调可能被拆成多段，必须累计。

HTMLParser：开始标签回调给出属性列表，文本由 `handle_data` 返回；需要跟踪当前所在结构，不应假设一次回调就是一个完整标题。

E43 urllib 获取并解析 JSON

题意改写 | 原 16.10：让 `fetch_data(url)` 返回 JSON 对应的 Python 对象。

```
import json
from urllib import request

def fetch_data(url):
    req = request.Request(url, headers={'Accept': 'application/json'})
    with request.urlopen(req, timeout=10) as response:
        charset = response.headers.get_content_charset() or 'utf-8'
        return json.loads(response.read().decode(charset))

# Offline verification; no external API key is required:
url = 'data:application/json,%7B%22city%22%3A%22Beijing%22%7D'
assert fetch_data(url) == {'city': 'Beijing'}
```

详解： 用 with 释放响应资源；`read` 得到字节，按响应编码解码，再 `json.loads`。原天气 API 的密钥与配额是外部条件，因此不把网页中公开的旧密钥写进笔记；如需访问天气服务，使用自己的有效配置。HTTP 错误或非法 JSON 让异常暴露给调用者处理。

E44 用 Expat 解析天气 XML

题意改写 | 原 16.11：提取城市、天气描述、温度与风速。示例按 WeatherAPI 的标签结构构造，可离线验证。

```
from xml.parsers.expat import ParserCreate

def parseXml(xml_str):
    parser = ParserCreate()
    stack, buffers, values = [], [], {}

    def start(name, attrs):
        stack.append(name)
        buffers.append([])

    def chars(data):
        if buffers:
            buffers[-1].append(data)

    def end(name):
        path = tuple(stack)
        values[path] = ''.join(buffers.pop()).strip()
        stack.pop()

    parser.StartElementHandler = start
    parser.CharacterDataHandler = chars
    parser.EndElementHandler = end
    parser.Parse(xml_str, True)
    return {
        'city': values[('root', 'location', 'name')],
        'weather': {
            'condition': values[('root', 'current', 'condition', 'text')],
            'temperature': float(values[('root', 'current', 'temp_c')]),
            'wind': float(values[('root', 'current', 'wind_kph')])
        }
    }

xml = ('<root><location><name>Beijing</name></location><current>'
       '<temp_c>37.2</temp_c><wind_kph>9.7</wind_kph>'
       '<condition><text>Sunny</text></condition></current></root>')
result = parseXml(xml)
assert result == {'city': 'Beijing', 'weather': {
    'condition': 'Sunny', 'temperature': 37.2, 'wind': 9.7}}
```

详解：标签栈区分嵌套位置；每个标签独立累计文本，结束标签时保存完整路径与字符串；温度、风速再转数值。不能把字符回调的最后一次片段直接当作最终文本。此实现对应给定标签结构，带 XML 命名空间或不同服务的结构需另做路径映射。

E45 HTMLParser 提取会议名称、时间和地点

题意改写 | 原 16.12：对照 Python 官网会议列表的常见 HTML 结构提取信息。

```
from html.parser import HTMLParser

class EventParser(HTMLParser):
    def __init__(self):
        super().__init__()
        self.events, self.current = [], None
        self.depth, self.field_depth, self.field = 0, 0, None

    def handle_starttag(self, tag, attrs):
        attrs = dict(attrs)
        classes = attrs.get('class', '').split()
        if tag == 'li':
            self.depth += 1
            if self.current is None:
                self.current = {'name': '', 'time': '', 'location': ''}
                self.item_depth = self.depth
        if self.current is None:
            return
        if 'event-title' in classes:
            self.field = 'name'
        elif 'event-location' in classes:
            self.field = 'location'
        elif tag == 'time':
            self.field = 'time'
        else:
            if self.field:
                self.field_depth += 1
            return
        self.field_depth = 1

    def handle_data(self, data):
        if self.current is not None and self.field:
            self.current[self.field] += data

    def handle_endtag(self, tag):
        if self.field:
            self.field_depth -= 1
            if self.field_depth <= 0:
                self.field = None
        if tag == 'li':
            if self.current is not None and self.depth == self.item_depth:
                event = {k: ' '.join(v.split())
                         for k, v in self.current.items()}
                if all(event.values()):
                    self.events.append(event)
                self.current = None
            self.depth = max(0, self.depth - 1)
```

E45（续）离线验证与详解

将下段接在解析器类的后面运行。先用固定 HTML 样本验证字段收集，再换成从网页读取的文本。

```
sample = ('<ul><li><h3 class="event-title"><a>Py Demo</a></h3>'
          '<time>2026-11-09</time>'
          '<span class="event-location">Kuala Lumpur</span></li></ul>')
p = EventParser()
p.feed(sample)
assert p.events == [{'name': 'Py Demo', 'time': '2026-11-09',
                     'location': 'Kuala Lumpur'}]
for event in p.events:
    print(event['time'], event['name'], event['location'])
```

详解：在列表项内跟踪三个字段；标题常嵌在 a 标签中，因此使用层级计数保持文本收集；`handle_data` 可能触发多次，采用追加。真实网页可先用 urllib 读取并解码，再 `p.feed(html)`；建议先定位会议列表区域，以免导航中的 li 干扰。示例数据是新编离线样本，并非某场真实会议的公告。

> **易错** 这个解析器对应示例的配对标签结构，不是通用 DOM 选择器。网页结构改动、字段内插入 br 等空元素时，需同步调整层级逻辑或使用适合复杂页面的 HTML 解析库。

K33 venv 与环境复现

`venv` 为项目隔离解释器环境和第三方依赖；激活环境主要修改当前终端的 PATH，不是切换系统 Python。本机可用以下命令创建环境并安装本笔记应用部分的依赖。

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install Pillow requests chardet psutil
python -m pip install flask aiohttp mysql-connector-python 'SQLAlchemy>=2,<3'
python -m pip freeze > requirements.txt
deactivate
```

解析：Windows 常用 `.venv\Scripts\activate` 或对应 PowerShell 脚本；未激活时也可直接调用环境内的解释器。记录版本文件帮助复现，但跨平台时还需检查 Python 版本与系统依赖。

> **验收** 能解释网络返回值为什么不能直接当 dict 使用，并能说明 XML 字符数据为什么需要累计。

15 第三方库与图形界面

来源章节：17 常用第三方模块、17.1 Pillow、17.2 requests、17.3 chardet、17.4 psutil；18 图形界面、18.1 海龟绘图。

K34 按任务选择库

- Pillow：用 `Image.open` 读取图像，`thumbnail` 原地缩略且保持比例，`resize` 返回指定尺寸的新图像；滤镜由 `ImageFilter` 提供；绘图由 `ImageDraw` 提供。保存 JPEG 前通常需要 RGB 模式。
- requests：`get/post` 发 HTTP 请求；`params` 编查询，`data` 常用于表单，`json` 编 JSON 请求体；`r.json()` 解析返回体，`raise_for_status()` 检查 HTTP 错误，始终设置适当 timeout。
- chardet：猜测字节编码，并给出 confidence；猜测不等于正确，短文本和多种兼容编码可能无法可靠区分。优先使用协议或文件规范指定的编码。
- psutil：CPU、内存、磁盘、网络、进程监测。`cpu_percent` 首次调用与测量间隔有关；进程可能随时退出或无权访问，要处理 `NoSuchProcess/AccessDenied`。
- Tkinter：窗口、控件和回调由事件循环驱动；`command=handler` 传函数，不是 `command=handler()`；长任务不能直接堵住界面事件处理。
- turtle：以朝向和步长绘图；角度控制转向，`done()` 保持窗口。运行依赖可用的桌面和 Tk 环境。

S16 Pillow 缩略图与滤镜

原 17.1 示例整理｜需先安装 Pillow，输入图像自行提供；本笔记不生成或改动用户图片。

```
from PIL import Image, ImageFilter

def make_thumbnail(input_path, output_path):
    with Image.open(input_path) as image:
        image = image.convert('RGB')
        image.thumbnail((320, 240))
        image = image.filter(ImageFilter.BLUR)
        image.save(output_path, 'JPEG')

# make_thumbnail('input.jpg', 'thumbnail.jpg')
```

详解：先转为 RGB 以适配 JPEG，再缩略并应用模糊滤镜。宽高上限是 320×240，但保持比例后不一定正好等于这个尺寸。滤镜调用返回新图像，需要接住返回值。

S17 requests 获取 JSON 与资源监控

```
import requests

def get_json(url):
    response = requests.get(url, timeout=(3, 10))
    response.raise_for_status()
    return response.json()

# data = get_json('https://example.com/api/data')
```

详解：该 URL 只是接口位置示意，需换成返回 JSON 的真实服务；`timeout` 的两个值分别控制连接与读取等待，不代表整个请求必然在 13 秒内结束。

```
import chardet
import psutil

raw = '中文'.encode('utf-8')
print(chardet.detect(raw))
print('CPU:', psutil.cpu_count())
print('memory:', psutil.virtual_memory().percent)
for proc in psutil.process_iter(['pid', 'name']):
    try:
        print(proc.info)
    except (psutil.NoSuchProcess, psutil.AccessDenied):
        continue
```

详解：编码检测输出可能随库版本、样本变化，不应把某个置信度硬编码为唯一答案。进程列表是瞬时观察，并非固定结果。

S18 Tkinter 输入与海龟绘图

原 18、18.1 示例整理｜两段分别保存运行，关闭窗口后程序结束。

```
import tkinter as tk
from tkinter import messagebox

def main():
    root = tk.Tk()
    root.title('Greeting')
    entry = tk.Entry(root)
    entry.pack(padx=20, pady=10)
    def hello():
        name = entry.get().strip() or 'world'
        messagebox.showinfo('Hello', f'Hello, {name}!')
    tk.Button(root, text='Greet', command=hello).pack(pady=10)
    root.mainloop()

if __name__ == '__main__':
    main()
```

详解：回调读取当前输入，不是在建按钮时读取。`mainloop()` 处理输入和重绘事件；没有它，界面可能刚创建就结束。

```
import turtle

def draw_star():
    pen = turtle.Turtle()
    for _ in range(5):
        pen.forward(120)
        pen.right(144)
    turtle.done()

if __name__ == '__main__':
    draw_star()
```

详解：五角星每段向右转 144°，五次共转 720°，形成交叉星形。若画正五边形，则外角为 72°，不能混用。

验收 能选择安装名和导入名，知道事件回调为什么传函数对象，并能解释编码检测只是推断。

16 TCP、UDP 与电子邮件

来源章节：19 网络编程、19.1 TCP/IP 简介、19.2 TCP 编程、19.3 UDP 编程；20 电子邮件、20.1 SMTP 发送邮件、20.2 POP3 收取邮件。

K35 TCP/IP 与 socket

IP 负责寻址，端口区分一台主机上的服务。TCP 是可靠、有序的字节流；UDP 面向数据报，不保证到达、顺序和去重。TCP 客户端 connect；服务端 bind、listen、accept，之后对新连接收发；UDP 常用 sendto/recvfrom，无需建立相同的连接过程。

重点 TCP 没有应用消息边界。一次 send 不保证对应一次 recv；接收可能拆开或合并。应使用固定长度、分隔符或长度前缀建立协议；`sendall()` 负责发送完整字节串，`recv()` 返回空字节表示对方关闭。

S19 带换行协议的本地 TCP Echo

原 19.2 示例整理 | 先运行服务端，再运行客户端。服务端处理一条连接、一行消息后退出。

```
# tcp_server.py
import socket

def main():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
        server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        server.bind(('127.0.0.1', 19001))
        server.listen(1)
        conn, address = server.accept()
        with conn:
            conn.settimeout(10)
            with conn.makefile('rb') as stream:
                line = stream.readline(4096)
                if not line.endswith(b'\n'):
                    raise ValueError('missing delimiter or line too long')
                conn.sendall(b'Hello, ' + line)

if __name__ == '__main__':
    main()
```

```
# tcp_client.py
import socket

with socket.create_connection(('127.0.0.1', 19001), timeout=5) as conn:
    conn.sendall(b'Python\n')
    with conn.makefile('rb') as stream:
        print(stream.readline(4096).decode('utf-8').rstrip())
```

详解：协议约定一条消息以换行结束，因此 `readline` 能跨多次底层接收合并数据，客户端输出 `Hello, Python`。上限 4096 防止无限读；服务端等待一条连接是演示限制，不是完整并发服务器。

S20 UDP 请求与响应

```
# udp_server.py -- run first
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as server:
    server.bind(('127.0.0.1', 19002))
    data, address = server.recvfrom(4096)
    server.sendto(b'Hello, ' + data, address)
```

```
# udp_client.py
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as client:
    client.settimeout(5)
    client.sendto(b'Python', ('127.0.0.1', 19002))
    data, address = client.recvfrom(4096)
    print(data.decode('utf-8'))
```

详解：服务端收到的是一份数据报及来源地址，再向该地址回复。超时并不自动重发；需要可靠交互时，应设计序号、重试和去重或直接选择更适合的协议。

K36 邮件传输与 MIME

- 邮件客户端通过 SMTP 提交邮件；POP3 用于下载邮箱中的邮件，IMAP 可用于在服务器上管理和同步，原教程聚焦 POP3。
- 邮件头与信封是两个层次；`From/To/Subject` 属于消息内容，SMTP 的实际收件地址由发送调用指定。中文标题和地址显示名需要正确的邮件编码处理。
- MIME 支持纯文本、HTML、附件和内嵌图片；`multipart/alternative` 提供同一内容的不同格式；附件采用合适的 Content-Type 和文件名，内嵌图通过 Content-ID 引用。
- 加密连接常用 `SMTP_SSL/POP3_SSL` 或服务支持的 STARTTLS；服务器地址、端口、认证方式与应用密码以自己的服务配置为准。

S21 构造多格式邮件与附件

原 20.1 示例整理丨只构造消息，不实际发送。

```
from email.message import EmailMessage

msg = EmailMessage()
msg['From'] = 'sender@example.com'
msg['To'] = 'recipient@example.com'
msg['Subject'] = 'Python 复习笔记'
msg.set_content('你好！附件是示例文本。')
msg.add_alternative('<p> 你好！<b> 附件</b> 是示例文本。</p>', subtype='html')
msg.add_attachment(b'hello', maintype='text', subtype='plain',
                   filename='note.txt')
assert msg.is_multipart()
```

详解：`EmailMessage` 自动组织 MIME 层次和头部编码，避免手工拼接。示例地址使用保留的 example.com，不能当作真实账户。

```
import os
import ssl
import smtplib

def send_message(msg):
    context = ssl.create_default_context()
    with smtplib.SMTP_SSL(os.environ['SMTP_HOST'], 465,
                          context=context, timeout=10) as server:
        server.login(os.environ['SMTP_USER'], os.environ['SMTP_PASSWORD'])
        server.send_message(msg)

# Call only after configuring a real account and intended recipient.
```

详解：发送函数展示 SSL 连接、登录、发送与自动退出；真实服务若要求其他端口或 STARTTLS，需要按该服务文档调整。本笔记未执行发送，也不把凭据写进代码。

S22 POP3 接收与解析邮件

原 20.2 示例整理丨函数实际连接时需有效账户；读取后不调用 dele，因此不主动删除服务器邮件。

```
import poplib
import ssl
from email import policy
from email.parser import BytesParser

def receive_last(host, username, password):
    server = poplib.POP3_SSL(host, 995, timeout=10,
                             context=ssl.create_default_context())
    try:
        server.user(username)
        server.pass_(password)
        count, size = server.stat()
        if count == 0:
            return None
        response, lines, octets = server.retr(count)
        return BytesParser(policy=policy.default).parsebytes(
            b'\r\n'.join(lines))
    finally:
        server.quit()

raw = b'Subject: demo\r\nContent-Type: text/plain; charset=utf-8\r\n\r\nhello'
message = BytesParser(policy=policy.default).parsebytes(raw)
assert str(message['Subject']) == 'demo'
assert message.get_content() == 'hello'
```

详解：POP3 返回逐行字节，用 CRLF 拼回原始消息，再交给邮件解析器。复杂多部分邮件可用 `walk()` 遍历或 `get_body()` 选择正文，不应把附件内容直接当纯文本解码。

> **验收** 能说明 TCP 流为何需要协议边界、UDP 超时如何处理，以及 MIME、SMTP、POP3 各负责哪一层。

17 数据库、SQL 与 ORM

来源章节：21 访问数据库、21.1 使用 SQLite、21.2 使用 MySQL、21.3 使用 SQLAlchemy。

K37 连接、游标与事务

- 关系数据库按表组织数据，主键标识记录；NoSQL 是多类非关系存储的统称，不能简单等同于“不能查询”。
- Connection 管理连接与事务；Cursor 执行语句、读取结果。写入通常需要 `commit()`，失败时 `rollback()`；关闭资源与提交事务是不同操作。
- 查询无 `ORDER BY` 时不能依赖某种固定顺序；`WHERE` 过滤，`ORDER BY score ASC` 排序；`BETWEEN` 包含两端。
- 参数化查询把 SQL 结构与参数分开。SQLite 常用 `?`，mysql-connector 常用 `%s`；占位符不是 Python 字符串格式化，不要自己把参数拼进 SQL。

E46 SQLite 按闭区间查询姓名

题意改写 | 原 21.1：按分数区间查询名字，结果按分数从低到高排列。

```
import sqlite3
from contextlib import closing

db_file = 'practice.db'

def prepare_db():
    with closing(sqlite3.connect(db_file)) as conn:
        with conn:
            conn.execute('CREATE TABLE IF NOT EXISTS user '
                         '(id TEXT PRIMARY KEY, name TEXT, score INTEGER)')
            conn.executemany('INSERT OR REPLACE INTO user VALUES (?, ?, ?)',
                             [('A-001', 'Adam', 95),
                              ('A-002', 'Bart', 62),
                              ('A-003', 'Lisa', 78)])

def get_score_in(low, high):
    with closing(sqlite3.connect(db_file)) as conn:
        rows = conn.execute(
            'SELECT name FROM user WHERE score BETWEEN ? AND ? '
            'ORDER BY score ASC, id ASC', (low, high)).fetchall()
    return [row[0] for row in rows]

prepare_db()
assert get_score_in(80, 95) == ['Adam']
assert get_score_in(60, 80) == ['Bart', 'Lisa']
assert get_score_in(60, 100) == ['Bart', 'Lisa', 'Adam']
```

详解：闭区间条件包含 low、high；排序先按 score，再按 id 稳定处理同分。`fetchall()` 每行是元组，列表生成式只抽取姓名。`closing` 确保真正关闭连接；SQLite 连接自身的 `with conn:` 管事务，不会单独负责关闭连接。练习采用独立数据库文件，不删除原有数据库；合并原题时沿用原来的 `db_file` 与初始化代码。

S23 MySQL 参数化访问

原 21.2 示例整理 | 需要 MySQL 服务、数据库 test，以及 mysql-connector-python。

```
import os
import mysql.connector

def lookup_user(user_id):
    conn = mysql.connector.connect(
        host='127.0.0.1', user=os.environ['MYSQL_USER'],
        password=os.environ['MYSQL_PASSWORD'], database='test')
    try:
        cursor = conn.cursor()
        try:
            cursor.execute('SELECT id, name FROM user WHERE id=%s',
                           (user_id,))
            return cursor.fetchall()
        finally:
            cursor.close()
    finally:
        conn.close()
```

详解：即使只有一个参数，也要写 `(user_id,)`。查询依赖已存在的 user 表；这里只读取，写操作还需明确事务提交和回滚。连接数据库失败时，不应继续使用尚未成功建立的 cursor。

K38 SQLAlchemy 的对象映射

ORM 用类映射表、属性映射列、实例映射行。Engine 管理数据库连接基础设施；Session 是工作单元，跟踪对象变化并提交事务；relationship 表达对象间关系。SQLAlchemy 的 `Column`/基类等写法在旧教程常见，下面给出 2.x 风格的明确版本示例。

S24 SQLAlchemy 2.x 最小示例

```
from sqlalchemy import create_engine, String, select
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = 'users'
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(30))

engine = create_engine('sqlite:///:memory:')
Base.metadata.create_all(engine)
with Session(engine) as session:
    session.add(User(id=1, name='Alice'))
    session.commit()
    user = session.scalars(select(User).where(User.id == 1)).one()
    assert user.name == 'Alice'
```

详解：示例用内存 SQLite，避免要求 MySQL 服务；`create_all` 创建缺失表，不是完整迁移系统；`add` 先加入工作单元，commit 后持久化。`.one()` 要求恰好一行，零行或多行均抛出异常；应根据业务选择 `.first()`、`.one_or_none()` 等结果约定。

验收 能解释 commit、close、fetchall 的区别，并能写出不拼接用户输入的区间查询。

18 HTTP、HTML、WSGI 与模板

来源章节：22 Web 开发、22.1 HTTP 协议简介、22.2 HTML 简介、22.3 WSGI 接口、22.4 使用 Web 框架、22.5 使用模板。

K39 Web 请求路径

浏览器发出 HTTP 请求，服务器根据方法和路径选择处理逻辑，返回状态、响应头和响应体。常用 GET 获取资源、POST 提交数据；HTTPS 在 HTTP 传输上使用 TLS，不能用 GET/POST 的选择替代传输加密。

HTML 描述结构，CSS 控制样式，JavaScript 提供浏览器端行为。表单 name 决定提交的字段名；模板在服务端把数据填入页面。业务逻辑、页面结构与数据访问分离，便于测试与维护。

重点 HTTP/1.x 报文的头部与正文之间有空行；原始 socket 读取时必须考虑流式边界与编码。实际应用优先使用框架或客户端库，不手工拼出全部协议细节。

K40 WSGI 与框架分工

WSGI 是 Python Web 应用与服务器之间的接口约定。应用签名通常是 `application(environ, start_response)`：从 environ 读取请求，调用 `start_response` 设置状态与头部，返回产生 bytes 的可迭代对象。WSGI 不是网站框架，也不是 HTTP 协议本身。

S25 最小 WSGI 应用

原 22.3 示例整理丨运行后访问本机 8000 端口，Ctrl+C 退出。

```
from wsgiref.simple_server import make_server
from html import escape

def application(environ, start_response):
    name = environ.get('PATH_INFO', '/').lstrip('/') or 'world'
    body = f'<h1>Hello, {escape(name)}!</h1>'.encode('utf-8')
    start_response('200 OK', [
        ('Content-Type', 'text/html; charset=utf-8'),
        ('Content-Length', str(len(body)))
    ])
    return [body]

if __name__ == '__main__':
    with make_server('127.0.0.1', 8000, application) as server:
        server.serve_forever()
```

详解：HTTP 响应体必须是字节，因此先编码；Content-Length 是字节数，不是中文字符数。路径来自用户输入，嵌入 HTML 前做转义。这里的演示服务器适合本地学习，不代表完整部署方案。

S26 Flask 路由、表单与模板

原 22.4—22.5 示例整理丨需安装 Flask。HTML 内联以便一段代码独立运行；真实项目通常放入 templates 目录后使用 `render_template()`。

```
from flask import Flask, request, render_template_string

app = Flask(__name__)
FORM = '''<!doctype html><meta charset="utf-8">
<form method="post" action="/signin">
  <input name="username" placeholder="Name">
  <button type="submit">Sign in</button>
</form>'''
WELCOME = '<!doctype html><meta charset="utf-8"><h1>Hello, {{ name }}!</h1>'

@app.get('/')
def home():
    return FORM

@app.post('/signin')
def signin():
    name = request.form.get('username', '').strip()
    if not name:
        return 'name required', 400
    return render_template_string(WELCOME, name=name)

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=5001)
```

详解：GET 路由显示表单，POST 路由从 `request.form` 读取提交值；字段名与 input 的 name 对应。模板通过 `{{ name }}` 填入数据，默认转义；不要把用户输入拼成模板源码，也不要随意标为 safe。示例只问候输入的名字，没有密码验证、会话或实际登录系统。

> **验收** 能把“浏览器 →HTTP→ 服务器 →WSGI→ 框架路由 → 模板 → 响应”完整讲通，并指出表单字段在哪里进入 Python。

19 协程、asyncio 与 aiohttp

来源章节：23 异步 IO、23.1 协程、23.2 使用 asyncio、23.3 使用 aiohttp。

K41 从生成器协作到 async/await

原教程用生成器的 `send()` 演示协作：先 `send(None)` 启动，再把数据送到 yield 表达式，yield 同时可向外返回结果。这有助于理解暂停和恢复；现代 asyncio 优先使用 `async def` 和 `await`，不要把旧 `@asyncio.coroutine` 形式照搬进新项目。

重点 调用 async 函数只得到协程对象，不会自动并发执行。`asyncio.run(main())` 运行顶层入口；`await` 等待可等待对象；`gather()` 或预先创建任务可调度多个工作。逐句 `await a(); await b()` 通常表示先后执行。

易错 在协程里调用阻塞的 `time.sleep()`、同步 requests 或长时间 CPU 循环，会阻塞事件循环。应选异步 IO，或把适合的阻塞工作交给 `asyncio.to_thread()` 等机制。即使单线程，任务也可能在 await 两侧交错修改状态，必要时仍需 `asyncio.Lock`；不能把“协程从不需要锁”当作一般结论。

S27 生成器 send 的生产者—消费者

原 23.1 示例整理。

```
def consumer():
    reply = ''
    while True:
        item = yield reply
        if item is None:
            return
        reply = f'processed {item}'

c = consumer()
assert c.send(None) == ''
assert c.send(1) == 'processed 1'
assert c.send(2) == 'processed 2'
c.close()
```

详解：首次 send(None) 执行到第一个 yield；之后 send(1) 的值成为 `item`，处理完成再次走到 yield 时交回回复。它是同步、显式的协作切换演示，不是已经自动接入 asyncio 的网络协程。

S28 asyncio 并发与结果顺序

原 23.2 示例整理。

```
import asyncio
import threading

async def hello(name):
    print('start', name, threading.current_thread().name)
    await asyncio.sleep(0.01)
    return f'Hello, {name}!'

async def main():
    results = await asyncio.gather(hello('Bob'), hello('Alice'))
    assert results == ['Hello, Bob!', 'Hello, Alice!']
    return results

if __name__ == '__main__':
    print(asyncio.run(main()))
```

详解：两个协程等待期间可交替运行，不需要两个 Python 线程；`gather` 的结果列表按参数顺序排列，而不是按完成先后排列。原教程部分打印写成 `threading.current_thread`，显示的是函数对象；这里补上括号取得实际线程信息。

S29 异步 HTTP 客户端与 aiohttp 服务器

原 23.2 网络示例的配套整理丨用 HTTP 库处理响应，避免手写 TCP HTTP 头解析；需 aiohttp 和可用网络。

```
import asyncio
import aiohttp

async def fetch_texts(urls):
    timeout = aiohttp.ClientTimeout(total=10)
    async with aiohttp.ClientSession(timeout=timeout) as session:
        async def fetch(url):
            async with session.get(url) as response:
                response.raise_for_status()
                return await response.text()
        return await asyncio.gather(*(fetch(url) for url in urls))

# asyncio.run(fetch_texts(['https://example.com']))
```

详解：复用一个 Session 管理连接；两个 async with 分别管理会话和响应资源；响应体读取也要 await。`gather` 不限制并发数量，URL 很多时需要连接池上限或 Semaphore。

```
# aiohttp_server.py
from aiohttp import web
from html import escape

async def index(request):
    return web.Response(text='<h1>Index Page</h1>',
                        content_type='text/html')

async def hello(request):
    name = escape(request.match_info['name'])
    return web.Response(text=f'<h1>Hello, {name}!</h1>',
                        content_type='text/html')

app = web.Application()
app.add_routes([web.get('/', index), web.get('/{name}', hello)])
if __name__ == '__main__':
    web.run_app(app, host='127.0.0.1', port=8080)
```

详解：框架将 URL 路径参数放入 `match_info`，再调用对应 async 处理函数。访问 `/Bob` 得到问候。处理函数是 async 并不自动使内部任何阻塞代码都变成非阻塞，仍要检查实际调用。

> **验收** 能说明 await 的暂停位置，判断两个任务是顺序等待还是并发调度，并解释资源为什么使用 async with。

20 FAQ、速查与自测回路

来源章节：24 FAQ、25 期末总结；结合全教程新编复习题。

K42 常用路径与解释器信息

```
import os
import sys
from pathlib import Path

print('working directory:', os.getcwd())
print('arguments:', sys.argv)
print('interpreter:', sys.executable)
if '__file__' in globals():
    print('module file:', Path(__file__).resolve())
    print('module folder:', Path(__file__).resolve().parent)
```

详解：工作目录由启动方式决定，不一定是脚本目录；`__file__` 在部分交互环境不存在；`sys.executable` 帮助排查“装了包但导入失败”是否源于解释器不一致。用项目实作检验知识，比仅反复阅读更能发现盲点。

十条必须能脱口而出的规则

- 变量赋值是名字绑定；可变对象别名要注意共享修改。
- 真除法 `/`、地板除 `//`、取余 `%`；地板除负数向下取整。
- `str` 是文本，`bytes` 是字节；编码向外，解码向内。
- 单元素元组靠逗号；空字典是 `{}`，空集合是 `set()`。
- 默认参数定义时求值；`*args` 收位置参数，`**kwargs` 收关键字参数。
- 可迭代不等于迭代器；生成器惰性计算，消费后通常不重来。
- 装饰器包装调用；保持返回值、异常和函数元信息。
- 文件资源用 with；事务提交和连接关闭是两个动作。
- TCP 是字节流，需要消息边界；SQL 参数单独绑定。
- async 函数调用返回协程对象；并发需要调度，阻塞 IO 不会自动变异步。

Q01—Q08 独立自检（新编题）

先遮住后面的答案。每题要求同时写出结论和一句理由；不会的知识点仍标为“待验证”。

- Q01：`a=[[]]; b=a.copy(); b[0].append(1)` 后 a 是什么？
- Q02：`-8 // 3`、`-8 % 3` 分别是多少？
- Q03：为什么 `def f(L=[]): ...` 可能使两次无参调用互相影响？
- Q04：`g=(x*x for x in range(3)); list(g); list(g)` 第二次是什么？

- Q05：写出一个单次赋值可用、外部普通赋值会失败的只读 property 思路。
- Q06：SQL 查询要按分数排序并包含 60、80 两个端点，应包含哪些子句？
- Q07：先 `await a()` 再 `await b()` 与 `await gather(a(),b())` 的调度有什么区别？
- Q08：拿到网络 bytes 后，要得到 JSON 的 dict，通常经过哪两步？

自检答案与迁移

Q01：`[[1]]`。只复制了外层，内部列表仍共享。迁移：嵌套配置、矩阵初始化同样要检查共享引用。

Q02：-3、1。向负无穷取整，并满足除法恒等式。

Q03：默认列表在定义时只创建一次，被后续调用复用。改用 None 并在函数内新建。

Q04：空列表 `[]`。同一生成器已耗尽；需要重新生成表达式才能从头开始。

Q05：只定义 `@property def value(self): return self._value`，在构造函数设置 `_value`，不提供 value.setter。它限制常规属性接口，不是不可绕过的安全机制。

Q06：`WHERE score BETWEEN ? AND ? ORDER BY score ASC`，参数 `(60,80)`；若要求同分确定顺序，再加第二排序键。

Q07：前者通常先把 a 完整等待结束再启动 b；后者把两个协程一起调度，允许在可等待点交错执行。

Q08：按正确字符集 decode 成 str，再 `json.loads` 成 Python 对象；也可使用库提供的 JSON 接口封装这些步骤。

不依赖考试日期的复习安排

这是一套可重复的建议回路，不是按真实考试日期排定的备考计划。首次复习先按 K01—K13 验收核心语法；再按 K14—K25 验收程序组织、IO 和异常；最后用 K26—K42 验收库与应用。每轮把不看答案能独立写出的题标为“自测通过”，出错题记录“题号—原作答—具体错步—订正规则—下次复做结果”。

建议每个主题完成后，在第 1、3、7 天重新做一题同构题；若连续两次仍在相同边界失败，回到对应知识点补写一个最小反例，再做条件改变的变式。不把本文给出的演示测试当作你的个人掌握记录。

附录 A 练习覆盖与来源索引

本笔记 E01—E46 共 46 组答案，覆盖原教程 40 个带“练习”栏的页面及其多个小问；另有 S01—S29 的讲解示例整理和配套补充。原题题意均为改写，代码答案为本笔记编写或重组；不冒称为作者提供的唯一官方答案。

原练习到本笔记的映射

- 4.2→E01；5.1→E02；5.2→E03；5.3→E04（嵌套索引、元组选择）；5.4→E05；5.6→E06（含无限循环尝试）。
- 6.1→E07；6.2→E08；6.3→E09；6.4→E10。
- 7.1→E11；7.2→E12；7.3→E13；7.4→E14。
- 8.1.1→E15—E17；8.1.2→E18；8.1.3→E19（两种排序）；8.2→E20；8.3→E21；8.4→E22—E23（三种装饰需求）。
- 10.2→E24；10.5→E25；11.2→E26；11.5→E27。
- 12.1→E28；12.3→E29；12.4→E30。
- 13.1→E31；13.3→E32—E33；13.4→E34。
- 15→E35—E36；16.1→E37；16.4→E38；16.6→E39—E40（含两种盐形式）；16.7→E41；16.8→E42；16.10→E43；16.11→E44；16.12→E45；21.1→E46。

版本校核与补充原始文档

- Python threading（GIL、线程异常与锁）：https://docs.python.org/3/library/threading.html
- Python hashlib（摘要和密码派生用途）：https://docs.python.org/3/library/hashlib.html
- Python asyncio（协程、任务与调度）：https://docs.python.org/3/library/asyncio-task.html
- SQLAlchemy 2.0 ORM 入门（本笔记明确采用 2.x 示例）：https://docs.sqlalchemy.org/en/20/orm/quickstart.html

本笔记的完整性与运行界限

知识点摘要覆盖整套目录，但不是原教程逐段转载；入门导言合并，讲解中重复出现的同类演示用代表性示例说明。原页面练习栏的各小问已映射至 E 题号。网络例题提供离线数据，以区分“解析逻辑正确”与“某个远程 API 当时可用”。GUI、邮件账户、MySQL 等环境依赖的示例均写明前提。

验证报告由同目录源文件中的代码检查记录生成；文末紧接实际检查结果，不把未执行的外部操作写为已通过。PDF 为 LaTeX 源文件经 XeTeX 引擎编译，保留可重新编译的源文件。文件背景为白色，重点色块仅用于辅助阅读。

附录 B 代码验证记录

验证环境：3.12.14 / darwin

共 87 个 Python 代码块通过语法检查；78 个离线代码块完成执行；9 个依赖交互、网络或外部服务的代码块未执行完整集成流程。

原练习答案 E01—E46 全部通过独立核对：118 项语义检查通过；包括从原页面提取的可独立运行断言。

脚本执行在临时目录完成，未连接真实邮箱，未发送邮件，未访问 MySQL 账户，未启动 GUI。服务端与客户端示例通过语法核对；并发通信和第三方框架的实际验证范围以本页记录为准。

原站测试复用：20 组断言或测试尾段通过，其中包含 Student 的原始 4 个单元测试。

附加执行：线程锁计数、WSGI 应用响应、Flask 表单路由与 HTML 转义、asyncio 并发结果、SQLAlchemy 2.x 内存 SQLite 示例。

未做真实账户或外网集成：SMTP/POP3、MySQL 服务、天气 API；GUI 窗口与双端 socket/Manager 示例仅检查语法和代码组织。

检查边界 代码检查证明文中列明的行为；不把远程 API、用户账户或特定桌面环境的可用性写为已验证。

附录 C 全章节原文链接

以下条目对应 2026-11-09 读取到的原站目录。点击章节名可直接打开原文。教程原作者：廖雪峰。

1. 简介
2. Python 历史
3. 安装 Python
3.1. Python 解释器
4. 第一个 Python 程序
4.1. 使用文本编辑器
4.2. 输入和输出
5. Python 基础
5.1. 数据类型和变量
5.2. 字符串和编码
5.3. 使用 list 和 tuple
5.4. 条件判断
5.5. 模式匹配
5.6. 循环
5.7. 使用 dict 和 set
6. 函数
6.1. 调用函数
6.2. 定义函数
6.3. 函数的参数
6.4. 递归函数
7. 高级特性
7.1. 切片
7.2. 迭代
7.3. 列表生成式
7.4. 生成器
7.5. 迭代器
8. 函数式编程
8.1. 高阶函数
8.1.1. map/reduce
8.1.2. filter
8.1.3. sorted
8.2. 返回函数
8.3. 匿名函数
8.4. 装饰器
8.5. 偏函数
9. 模块
9.1. 使用模块
9.2. 安装第三方模块
10. 面向对象编程
10.1. 类和实例
10.2. 访问限制
10.3. 继承和多态
10.4. 获取对象信息

10.5. 实例属性和类属性
11. 面向对象高级编程
11.1. 使用 __slots__
11.2. 使用 @property
11.3. 多重继承
11.4. 定制类
11.5. 使用枚举类
11.6. 使用元类
12. 错误、调试和测试
12.1. 错误处理
12.2. 调试
12.3. 单元测试
12.4. 文档测试
13. IO 编程
13.1. 文件读写
13.2. StringIO 和 BytesIO
13.3. 操作文件和目录
13.4. 序列化
14. 进程和线程
14.1. 多进程
14.2. 多线程
14.3. ThreadLocal
14.4. 进程 vs. 线程
14.5. 分布式进程
15. 正则表达式
16. 常用内建模块
16.1. datetime
16.2. collections
16.3. argparse
16.4. base64
16.5. struct
16.6. hashlib
16.7. hmac
16.8. itertools
16.9. contextlib
16.10. urllib
16.11. XML
16.12. HTMLParser
16.13. venv
17. 常用第三方模块
17.1. Pillow
17.2. requests
17.3. chardet
17.4. psutil
18. 图形界面
18.1. 海龟绘图
19. 网络编程
19.1. TCP/IP 简介

19.2. TCP 编程
19.3. UDP 编程
20. 电子邮件
20.1. SMTP 发送邮件
20.2. POP3 收取邮件
21. 访问数据库
21.1. 使用 SQLite
21.2. 使用 MySQL
21.3. 使用 SQLAlchemy
22. Web 开发
22.1. HTTP 协议简介
22.2. HTML 简介
22.3. WSGI 接口
22.4. 使用 Web 框架
22.5. 使用模板
23. 异步 IO
23.1. 协程
23.2. 使用 asyncio
23.3. 使用 aiohttp
24. FAQ
25. 期末总结